

UNIVERZITA J. SELYEHO – SELYE JÁNOS EGYETEM

**FAKULTA EKONÓMIE A INFORMATIKY –
GAZDASÁGTUDOMÁNYI ÉS INFORMATIKAI KAR**

**IDENTIFIKÁCIA OBRAZU A PRENOS ÚDAJOV
S IDENTIFIKÁCIU NA ZÁKLADE HASH –
KÉPAZONOSÍTÁS ÉS ADATTOVÁBBÍTÁS HASH-ALAPÚ
AZONOSÍTÁSSAL**

Diplomová práca – Diplomadolgozat

Bc. Ozogány László

2026

UNIVERZITA J. SELYEHO – SELYE JÁNOS EGYETEM

FAKULTA EKONÓMIE A INFORMATIKY –

GAZDASÁGTUDOMÁNYI ÉS INFORMATIKAI KAR

IDENTIFIKÁCIA OBRAZU A PRENOS ÚDAJOV

S IDENTIFIKÁCIU NA ZÁKLADE HASH –

**KÉPAZONOSÍTÁS ÉS ADATTOVÁBBÍTÁS HASH-ALAPÚ
AZONOSÍTÁSSAL**

Diplomová práca – Diplomadolgozat

Študijný program:	Aplikovaná informatika
Tanulmányi program:	Alkalmazott informatika
Názov študijného odboru:	18. -informatika
Tanulmányi szak megnevezése:	18. -informatika
Vedúci záverečnej práce:	Mgr. Dávid Paksi, PhD.
Témavezető:	Mgr. Dávid Paksi, PhD.
Školiace pracovisko:	Katedra informatiky
Tanszék megnevezése:	Informatikai Tanszék

Bc. Ozogány László

Komárno, 2026



ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc Ladislav Ozogány
Študijný program: Aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: 18. - informatika
Typ záverečnej práce: Diplomová práca
Jazyk záverečnej práce: maďarský

Téma: Képezonosítás és adattovábbítás hash-alapú azonosítással

Anotácia: A projekt célja egy olyan rendszer kidolgozása, amelyben a képek nem közvetlenül kerülnek átvitelre, hanem azok hash-értéke alapján történik az azonosítás és a kommunikáció. A megoldás lényege, hogy a forrásrendszer egy képhez egyedi hash-t generál, és a célrendszer ez alapján a hash alapján beazonosítja vagy előállítja a megfelelő képet. A projekt során vizsgálatra kerül a különböző hash-algoritmusok teljesítménye, valamint a módszer gyakorlati alkalmazhatóságának vizsgálata.

Kľúčové

slová: hash, képezonosítás, adatátvitel, adatbiztonság, képtömörítés

Vedúci: Mgr. Dávid Paksi, PhD.
Katedra: KINF - Katedra informatiky
ZOŠP: prof. RNDr. Tibor Kmet', CSc.
Dátum schválenia: 15.10.2025


prof. RNDr. Tibor Kmet', CSc.
osoba zodpovedná za realizáciu študijného programu

Abstrakt

V tejto práci predstavujeme softvérový systém Babylon Image Archive, ktorý realizuje unikátny bitovo balený kódovací postup obrazu s použitím paletovej kvantizácie, a integruje ho s bezpečnou vrstvou šifrovania založenou na hesle, ako aj s komponentou generovania obrazu riadenou umelou inteligenciou. Cieľom nášho systému je demonštrovať, že je možné vytvoriť nezávislú reprezentáciu obrazu od tradičných obrazových formátov, ktorá sa dá prenášať a zdieľať vo forme jediného textového reťazca (Image Token), a je interpretovateľná aj bez hesla – za predpokladu, že prijímateľ disponuje vhodným dekódovačom. Systém má štyri hlavné inovačné body:

- (1) unikátny formát BID (Babylon Image Data), ktorý reprezentuje obraz ako paletové indexy balené do 15-bitových segmentov a kompresiou zlib/lzma ho redukuje až na zlomok pôvodnej veľkosti dát;
- (2) dvojúrovňová kryptografická vrstva, ktorá kombinuje AES-128 šifrovanie (Fernet/PBKDF2) a maskovanie hash-u založené na HMAC-SHA256;
- (3) paletová kvantizácia založená na dátovej štruktúre scipy cKDTree, ktorá je bez explózie pamäte a garantuje deterministický čas behu pre všetky hodnoty values_per_segment (1–15);
- (4) integrácia difúzneho modelu s inferenciou na pozadí a automatickou detekciou hardvéru (CUDA/MPS/CPU).

Kľúčové slová:

hash, identifikácia obrazu, prenos dát, bezpečnosť dát, kompresia obrazu

Absztrakt

Jelen munkánkban bemutatjuk a Babylon Image Archive szoftverrendszert, amely egy egyedi, bit-csomagolt képkódolási eljárást valósít meg palettaalapú kvantálással, és ezt integrálja egy biztonságos, jelszóalapú titkosítási réteggel, valamint egy mesterséges intelligenciával vezérelt képgenerálási komponenssel. Rendszerünk célja demonstrálni, hogy hagyományos képformátumoktól független, önálló képreprezentáció hozható létre, amely egyetlen szöveges karakterlánc (Image Token) formájában hordozható, megosztható, és jelszó nélkül is értelmezhető — feltéve, hogy a fogadó rendelkezik a megfelelő dekódolóval. A rendszer négy fő innovációs pontja:

- (1) az egyedi BID (Babylon Image Data) formátum, amely 15 bites szegmensekbe csomagolt palettaindexekként reprezentálja a képet és zlib/lzma tömörítéssel akár az eredeti adatméret töredékére csökkenti azt;
- (2) egy kétszintű kriptográfiai réteg, amely AES-128 titkosítást (Fernet/PBKDF2) és HMAC-SHA256 alapú hash-maszkolást kombinál;
- (3) a scipy cKDTree adatstruktúrán alapuló, memóriarobbanástól mentes palettakvantálás, amely az összes values_per_segment értéknél (1–15) deterministikusan futásidőt garantál;
- (4) egy diffúziós modell integrációja háttérzúzás inferenciával és automatikus hardverdetektálással.

Kulcsszavak:

hash, képezonosítás, adatátvitel, adatbiztonság, képtömörítés

Tartalom

Bevezetés	8
1. Motiváció	9
1.1 Dolgozat Felépítése.....	11
2. Célkitűzések és Problémafelvetés.....	14
2.1. Fő Célkitűzések	15
2.2. Problémafelvetés	17
2.3 Ábrák és pszeudokód szerepe a módszertanban	18
2.4 A problémák és célok kapcsolata.....	19
3. Irodalmi Áttekintés.....	20
3.1. Hash-Függvények és Kriptográfiai Primitívek	23
3.2. Palettaalapú Képkódolás és Kvantálás	24
3.3. Tömörítési Algoritmusok: zlib és lzma	25
3.4. Diffúziós Modellek	26
3.5. Kapcsolódó Rendszerek és Előzmények.....	26
4. Módszertan	27
4.1 Kutatásmódszertan és fejlesztési keret.....	27
4.2 Hash-algoritmusok összehasonlítása	28
4.3 A képgenerálás folyamata	29
4.3.1 Eljárásos generálás hash alapján.....	30
4.3.2 AI-alapú generálás hash + prompt alapján	30
4.3.3 Kombinált megközelítés lehetősége	31
4.4 Értékelési metrikák.....	31
4.5 Tesztadathalmazok és benchmark környezet.....	32
4.6 Etikai és jogi szempontok a módszertanban.....	33
4.7 Reprodukálhatóság biztosítása	33
5. Rendszertervezés	34
5.1. Moduláris Architektúra	37
5.2. Adatfolyamok	38
5.3. Állapotmenedzsment.....	39
5.4. Biztonsági Architektúra	40
5.5. Felhasználói Felület Architektúrája.....	40
6. Implementáció	41
6.1 Fejlesztési környezet és rendszerfeltételek.....	41
6.2 Rendszertelepítés és indítási folyamat.....	42
6.3 Kulcskomponensek implementációja és kódlemezése	49

6.3.1 AI szerver működése.....	51
6.3.2 Képkódolás és bit-csomagolás.....	52
6.3.3 Hash-alapú reprezentáció.....	52
6.4 Optimalizációs technikák	53
6.5 Komponensek közötti integráció	55
6.6. A Kriptográfiai Modul.....	57
6.7. Az AI Komponens Implementációja	58
6.8 Összegzés.....	60
7. Tesztelés és Eredmények	61
7.1. Tesztelési Stratégia.....	61
7.2. BID Kerekutas Validáció.....	62
7.3. Teljesítménymérési Eredmények.....	62
7.4. Tömörítési Arányok.....	64
7.5. Vizuális Minőség — PSNR Összehasonlítás	65
8. Elemzés és Értékelés	66
8.1. Az Image Token Mechanizmus Értékelése	66
8.2. A Kriptográfiai Réteg Erőssége	66
8.3. Teljesítmény az Irodalomhoz Képest	67
8.4. Gyengeségek és Korlátozások	67
8.5. Összehasonlítás Kapcsolódó Rendszerekkel.....	68
9. Következtetés és Jövőbeli Fejlesztések.....	69
Resumé.....	71
Irodalomjegyzék.....	73

Bevezetés

A digitális korban az információáramlás és a multimédiás tartalmak kezelése alapvető szerepet játszik mindennapi életünkben és a technológiai alkalmazásokban. Különösen a képek, mint vizuális adatok, kulcsfontosságúak a kommunikációban, a szórakoztatásban, az oktatásban és az üzleti folyamatokban. A közösségi média platformoktól kezdve a távmunka-eszközökön át az adatbiztonsági rendszerekig számos területen találkozunk képek átvitelével és feldolgozásával. Azonban a hagyományos módszerek, amelyekben a képeket közvetlenül fájlformátumban továbbítjuk (pl. JPEG, PNG), számos kihívással járnak. Ezek közé tartozik a jelentős sávszélesség-igény, ami különösen problémás gyenge hálózati kapcsolatok esetén, valamint a biztonsági kockázatok, mint az adatvesztés, a hamisítás vagy az illetéktelen hozzáférés. Például egy nagy felbontású kép átvitele akár több megabájtot is igényelhet, ami lassítja a kommunikációt és növeli a költségeket.

Ezen problémák megoldására a jelen diplomadolgozat egy innovatív megközelítést javasol: a képek hash-érték alapján történő azonosítását és kommunikációját. A hash-érték egy egyedi digitális ujjlenyomat, amelyet kriptográfiai algoritmusok (pl. SHA-256) segítségével generálunk a kép tartalmából.

Munkánk kiindulópontja a Library of Babel webalkalmazás konceptuális kerete, amelyre támaszkodva egy ékírással agyagtáblák szimbolizmusára épülő képparchívum-motort hoztunk létre, amelyet mi Python nyelvben fejlesztettünk, majd jelentősen kiterjesztettünk. A Babylon nevet és koncepcióját az ókori mezopotámiai írásbeliségből merítettük: ahogyan az ékírás tömören, kódolt jelekkel rögzített összetett információt, úgy a mi rendszerünk is kompakt, bit-csomagolt karakterláncokba sűríti a vizuális adatokat. A projekt fő kérdésfeltevése a következő volt: lehetséges-e egy képet oly módon kódolni, hogy az egyetlen megosztható szöveges karakterlánccá váljon amelyből — fájlcsere nélkül — az eredetihez hasonló reprezentáció állítható elő a rendszer dekódolási vagy generatív eljárása segítségével? Erre a kérdésre az Image Token mechanizmus ad választ, amelyet részletesen tárgyalunk majd a következő fejezetekben.

1. Motiváció

A kutatás motivációja több egymással szorosan összefüggő tényezőből ered, amelyek a modern digitális társadalom alapvető problémáira vezethetők vissza. Napjainkban az információk túlnyomó része digitális formában keletkezik, és ezen belül is a vizuális adatok – különösen a képek – domináns szerepet töltenek be. A kommunikáció jelentős része képek, képernyőfotók, dokumentumok és vizuális tartalmak formájában történik, legyen szó közösségi médiáról, üzleti kommunikációról vagy akár oktatási célokról. Ezzel párhuzamosan azonban egyre több probléma is megjelenik, amelyek indokoltá teszik egy új, hatékonyabb és biztonságosabb megközelítés kidolgozását.

Az első és talán legkézenfekvőbb motivációs tényező a sávszélesség és az adatforgalom robbanásszerű növekedése. A globális adatforgalom évről évre exponenciálisan nő, és ennek jelentős részét a multimédiás tartalmak teszik ki (Cisco, 2023). Egyetlen nagy felbontású kép mérete több megabájt is lehet, és amikor naponta milliók osztanak meg képeket különböző platformokon, ez hatalmas terhelést jelent a hálózati infrastruktúrára. Bár a jelenlegi rendszerek különböző tömörítési technikákat alkalmaznak, ezek sok esetben kompromisszumot jelentenek a minőség rovására. Jó példa erre a közösségi média platformok működése: amikor egy felhasználó egy képet feltölt vagy elküld egy üzenetküldő alkalmazásban, a rendszer gyakran automatikusan újratömöríti azt. Ennek eredményeként a kép minősége jelentősen romlik, részletek vesznek el, és a vizuális élmény degradálódik. Ez a jelenség különösen jól megfigyelhető, amikor egy képet több alkalommal továbbítanak egymás között – minden egyes újraküldés tovább rontja a minőséget.

A hash-alapú megközelítés ezzel szemben radikálisan más utat kínál: a teljes kép helyett elegendő egy kompakt, egyedi azonosítót – egy hash-t vagy token-t – továbbítani. Ez az azonosító lényegében a kép „digitális lenyomata”, amely alapján a fogadó oldalon a kép rekonstruálható vagy újragenerálható. Ez nemcsak az adatforgalmat csökkenti drasztikusan, hanem megszünteti a többszöri tömörítésből eredő minőségromlást is.

A második kulcsfontosságú motiváció az adatvédelem és biztonság kérdése. A digitális kommunikáció egyik legnagyobb problémája, hogy az adatok gyakran harmadik feleken keresztül haladnak át, és a felhasználók nem mindig rendelkeznek teljes kontrollal afelett, hogy ezek az adatok hogyan kerülnek tárolásra vagy feldolgozásra. Az elmúlt években több olyan eset is napvilágot látott, amikor üzenetküldő platformok adatkezelési gyakorlata megkérdőjelezhető volt (WhatsApp/Meta adatvédelmi per, 2021), vagy amikor

felhasználói adatokhoz illetéktelenek fértek hozzá. A felhasználók gyakran feltételezik, hogy privát beszélgetéseik teljes mértékben védettek, azonban a valóságban sok esetben metaadatok, sőt bizonyos körülmények között tartalmi információk is feldolgozásra kerülhetnek.

A hagyományos képfájlok továbbítása különösen érzékeny pont ebből a szempontból, hiszen maga a kép is hordozhat személyes vagy bizalmas információkat. Egy hash-alapú rendszer esetén azonban az eredeti kép nem kerül közvetlenül továbbításra. Ehelyett egy olyan azonosító kerül megosztásra, amelyből az eredeti adat nem rekonstruálható egyszerűen visszafejtéssel. Ez jelentősen csökkenti az adatlopás és az illetéktelen hozzáférés kockázatát. Továbbá a kriptográfiai hash függvények ütközés-ellenállása biztosítja, hogy két különböző kép ne rendelkezzen azonos azonosítóval, ami kritikus fontosságú a rendszer megbízhatósága szempontjából.

A harmadik motiváció a fenntarthatóság és a hatékonyság kérdése. A digitális infrastruktúra működtetése jelentős energiafelhasználással jár, különösen az adatközpontok és felhőszolgáltatások esetében (IEA, 2023). A nagy mennyiségű adat tárolása és folyamatos mozgatása nemcsak gazdasági, hanem környezeti szempontból is terhelő. Egy olyan rendszer, amely képes jelentősen csökkenteni az átvitt és tárolt adatmennyiséget, közvetlenül hozzájárulhat az energiafogyasztás csökkentéséhez. A hash-alapú megközelítés lehetővé teszi, hogy egy archívumban ne magukat a képeket tároljuk, hanem azok azonosítóit, és a képeket csak szükség esetén generáljuk újra. Ez különösen előnyös olyan környezetekben, ahol a hardver erőforrások korlátozottak, például IoT eszközökön vagy mobil platformokon.

Negyedik motivációs tényezőként a mesterséges intelligencia gyors fejlődése emelhető ki, amely új lehetőségeket nyit a kutatás számára. Az utóbbi években a diffúziós modellek és más generatív AI rendszerek forradalmasították a képgenerálás területét. Ezek a modellek képesek rendkívül realisztikus képeket előállítani pusztán szöveges leírások alapján. Amennyiben a hash értéket seed-ként használjuk, lehetőség nyílik arra, hogy determinisztikus módon generáljunk képeket – vagyis ugyanabból a bemenetből mindig ugyanaz az output jön létre. Ez új dimenziókat nyit meg a digitális tartalomkezelésben, hiszen a képek nem statikus objektumokként, hanem újragenerálható entitásokként kezelhetők.

Ez a megközelítés különösen hasznos lehet kreatív iparágakban, oktatásban, valamint olyan rendszerekben, ahol a reprodukálhatóság kulcsfontosságú. Például egy oktatási

platformon elegendő lehet egy azonosítót megosztani a hallgatókkal, akik saját eszközeiken generálják le ugyanazt a vizuális tartalmat. Hasonlóképpen digitális művészeti archívumokban is alkalmazható, ahol a művek nem fájlként, hanem algoritmikusan reprodukálható formában kerülnek tárolásra.

A fentiek alapján megfogalmazható a kutatás alaphipotézise: a BID (Babylon Image Data) reprezentáció mérete a nyers palettaindex-tömb méretéhez képest természetesen ismétlődő tartalmak esetén 90%-os vagy nagyobb méretcsökkentést eredményez, míg az adatbiztonság szintjét egyidejűleg emeli. A dolgozat célja ennek a hipotézisnek a vizsgálata és egy működő prototípus rendszer bemutatása, amely gyakorlati környezetben is alkalmazható megoldást kínál a modern digitális kihívásokra.

1.1 Dolgozat Felépítése

A jelen diplomadolgozat felépítése egy logikusan strukturált, egymásra épülő fejezetsorozatot követ, amelynek célja, hogy az olvasót fokozatosan vezesse végig a vizsgált probléma elméleti háttérétől kezdve a konkrét megvalósításon át egészen az eredmények értékeléséig és a következtetések levonásáig. A dolgozat szerkezete tudatosan került kialakításra annak érdekében, hogy mind a tudományos megalapozottság, mind pedig a gyakorlati alkalmazhatóság szempontjai megfelelő hangsúlyt kapjanak.

A bevezető fejezetet követően a második fejezet részletesen ismerteti a kutatás célkitűzéseit és a problémafelvetést. Ez a fejezet bemutatja azt a technológiai és gyakorlati környezetet, amely indokolja a hash-alapú képreprezentációs rendszer vizsgálatát. A fejezet kitér a hagyományos módszerek – például klasszikus képtömörítési algoritmusok és fájlalapú adattovábbítás – korlátaira, különös tekintettel a minőségromlásra, a sávszélesség-igényre és az adatbiztonsági kockázatokra, és ezzel párhuzamosan megfogalmazza a dolgozat központi hipotézisét, valamint a javasolt megközelítés fő előnyeit, amelyek a hatékonyság, a biztonság és a reprodukálhatóság irányába mutatnak.

A harmadik fejezet átfogó irodalmi áttekintést nyújt, amely a téma tudományos megalapozását szolgálja. A fejezet részletesen tárgyalja a kriptográfiai hash-függvényeket, azok történeti fejlődését és gyakorlati alkalmazásait. A fejezet továbbá kitér az eljárásos tartalomgenerálás (procedural content generation) módszereire, amelyek alapot biztosítanak a determinisztikus képgenerálás megértéséhez. Külön hangsúlyt kapnak a modern mesterséges intelligencián alapuló képgeneráló modellek, például a diffúziós modellek, amelyek napjainkban a legfejlettebb megoldások közé tartoznak. Az irodalmi

áttekintés részeként bemutatásra kerülnek a perceptuális hash-elési technikák (például pHash), valamint a diffusion és más hasonló rendszerek működési elvei, összehasonlítva azok előnyeit és korlátait. A fejezet célja, hogy átfogó képet adjon a jelenlegi kutatási állapotról, és elhelyezze a dolgozatban bemutatott megoldást a meglévő technológiák kontextusában.

A negyedik fejezet a kutatás módszertanát részletezi. A fejezet ismerteti az alkalmazott fejlesztési megközelítést, amely tervezési-kísérleti megközelítésre épül: a független változók a `values_per_segment` paraméter (1–15) és a tömörítési algoritmus (`zlib/lzma/raw`); a függő változók a BID fájl méret, a futásidő és a vizuális minőség (PSNR). A hipotézis tesztelése 100 mintavételű benchmark futtatásokkal, `time.perf_counter()` pontossággal történt. A fejezet bemutatja a különböző hash-algoritmusok összehasonlítását, beleértve azok teljesítményét, ütközés-ellenállását és gyakorlati alkalmazhatóságát. Részletesen ismertetésre kerül a képgenerálás folyamata, különös tekintettel arra, hogy miként használható a hash érték seed-ként a determinisztikus generálás során. Emellett definiálásra kerülnek azok az értékelési metrikák is, amelyek alapján a rendszer teljesítménye mérhető, mint például a PSNR (Peak Signal-to-Noise Ratio), a generálási idő, valamint az adatsökkenés mértéke.

Az ötödik fejezet a rendszertervezésre fókuszál, amely a dolgozat egyik központi eleme. A fejezet bemutatja a teljes rendszer architektúráját, beleértve a főbb komponenseket és azok egymáshoz való viszonyát. A fejezet részletesen tárgyalja az egyes modulok feladatait, például a hash-generálásért felelős egységet, az AI-alapú képgeneráló modult, valamint a kommunikációs interfészeket. Konkrét példákon keresztül ismertetésre kerülnek a kulcsfontosságú fájlok és komponensek, mint például az „`ai_server.py`” és az „`secure.py`”, amelyek a rendszer működésének alapját képezik. Emellett a fejezet kitér a biztonsági megfontolásokra is, beleértve az adatvédelem, a hitelesítés és az integritás biztosításának kérdéseit.

A hatodik fejezet az implementáció részletes bemutatását tartalmazza. A fejezet ismerteti a fejlesztési környezet kialakítását, a szükséges szoftveres és hardveres feltételeket, valamint a rendszer telepítésének lépéseit. A fejezet részletes kódelemzéseket tartalmaz, amelyek a mellékelt forrásfájlokból származnak, és bemutatják a rendszer kulcsfontosságú működési mechanizmusait. Az implementáció során alkalmazott optimalizációs technikák, valamint a különböző komponensek közötti integrációs megoldások szintén részletesen bemutatásra kerülnek, biztosítva a rendszer hatékony és stabil működését.

A hetedik fejezet a tesztelési folyamatot és az elért eredményeket ismerteti. A fejezet bemutatja az egységteszteket, integrációs teszteket és teljesítményteszteket, amelyek a rendszer helyes működését és hatékonyságát igazolják. A fejezet táblázatok és grafikonok segítségével szemlélteti az eredményeket, különös tekintettel az adatcsökkentés mértékére, a generálási időkre és a képminőségre. Az eredmények összehasonlítása a hagyományos módszerekkel lehetőséget ad a javasolt megközelítés objektív értékelésére.

A nyolcadik fejezet az eredmények elemzését és értékelését tartalmazza. A fejezet kritikai szemlélettel vizsgálja a rendszer erősségeit és gyengeségeit, valamint azon tényezőket, amelyek befolyásolják a gyakorlati alkalmazhatóságot. A fejezet kitér a módszer korlátaira, például a generált képek pontosságára vagy a determinisztikus reprodukció feltételeire, valamint javaslatokat fogalmaz meg a továbbfejlesztés irányaira.

A dolgozatot a kilencedik fejezet zárja, amely összefoglalja a kutatás legfontosabb eredményeit és következtetéseit. A fejezet értékeli a hipotézist és felvázolja a jövőbeli fejlesztési lehetőségeket, amelyek további kutatási irányokat jelölnek ki. A dolgozat végén hivatkozások, idegen nyelvű összefoglalók (resumé és absztrakt), valamint mellékletek találhatók, amelyek tartalmazzák a teljes forráskódot, teszteseteket és egyéb kiegészítő anyagokat.

A fent ismertetett fejezeti struktúra biztosítja, hogy a dolgozat az elméleti háttér és a praktikus megvalósítás között egyensúlyt tartva, koherens és tudományos igényű módon tárgyalja a vizsgált problémakört.

2. Célkitűzések és Problémafelvetés

A kutatás központi célja egy olyan, módszertanilag megalapozott és technikailag koherens rendszer — a Babylon Image Archive, röviden BIA — kialakítása és elemzése, amely bit-csomagolt palettaalapú képkódolást, kétszintű kriptográfiai védelmet és AI-alapú képgenerálást integrál egyetlen egységes architektúrában. A célkitűzések meghatározásánál alapvető szempont volt, hogy az eredmények ne csupán elméleti hozzájárulásként jelenjenek meg. Hanem mérhetően és reprodukálhatóan demonstrálják a hash-alapú képreprezentáció gyakorlati értékét: csökkentett adatforgalmat, magasabb adatbiztonsági szintet és fájlcsere nélküli képmegoszthatóságot.

A vizsgált problémakör komplexitása indokoltá teszi a több szinten megfogalmazott célrendszer létrehozását. Elsődleges cél a terület alapfogalmainak, működési elveinek és a releváns technológiai háttérnek a rendszerezett feltárása, tudományos igényű definíciók és kategóriák kialakításával. Ezt követi a másodlagos célkitűzés: egy olyan modell vagy prototípus megalkotása, amely alkalmas a feltárt elméleti ismeretek gyakorlati validálására. A két célterület metszéspontjában helyezkedik el a harmadik kulcselem: a kísérleti eredmények, teljesítményadatok, statisztikai mutatók és összehasonlító elemzések alapján történő értékelés. Amely lehetővé teszi a rendszer előnyeinek, korlátainak és fejlesztési irányainak pontos meghatározását.

A célok kijelölését konkrét, mérhetően azonosított problémák motiválják: a nagy képfájlok korlátozott sávszélességű csatornákon való hatékony továbbításának igénye (pl. IoT-eszközök, mobil kommunikáció). Az érzékeny vizuális adatok (orvosi felvételek, digitális dokumentumok) biztonságos kezelésének követelménye, valamint a reprodukálható, fájlcsere nélküli képmegosztás lehetőségének megteremtése. A Babylon Image Archive ezekre a problémákra egyetlen, integrált architektúrával kínál választ, amelynek teljesítménye mérési adatokkal alátámasztható és tudományos eszközökkel értékelhető.

2.1. Fő Célkitűzések

A dolgozat kutatási céljainak meghatározása során törekedtünk arra, hogy a kitűzött célok egyszerre legyenek **elméleti érvényűek**, **gyakorlati haszonnal bírók**, valamint **mérhető módon validálhatók**. A fő célkitűzések több szintre tagolódnak, amelyek lefedik mind a háttérismeretek feltárását, mind az új rendszer megalkotásának lépéseit és kiértékelését.

1. Elméleti és technológiai háttér átfogó feltárása

Első célunk a Babylon Image Archive által lefedett problémakör — bit-csomagolt képkódolás, hash-alapú azonosítás, kriptográfiai védelem és diffúziós AI-generálás — megalapozott, rendszerezett leírása. A szakirodalmi áttekintés során külön hangsúlyt fektetünk:

- a releváns algoritmusok működési elveinek elemzésére,
- a különböző megközelítések közötti módszertani különbségekre,
- a történeti fejlődésre,
- a modern rendszerekben megjelenő kihívásokra és korlátokra.

Ez a háttér ismeri fel azokat a hiányosságokat, amelyek indokoltá teszik a dolgozat által javasolt megoldást.

2. Egy működő rendszer vagy modell megtervezése és implementálása

A célkitűzés második rétege a gyakorlati demonstrátor vagy modell létrehozása. A rendszer célja a vizsgált algoritmusok, struktúrák vagy technológiák alkalmazhatóságának bemutatása valós környezetben.

A fejlesztés során külön figyelmet kap:

- az architektúra moduláris kialakítása,
- a rendszer átlátható adatfolyamai,
- erőforrás-hatékony algoritmusok megvalósítása,
- megfelelő hibakezelési mechanizmusok,
- a folyamatok vizuális (ábrákkal) és algoritmikus (pszeudokódokkal) dokumentálása.

A prototípus kialakítása lehetőséget ad az elméleti kérdések gyakorlati tesztelésére és a hipotézisek értékelésére.

3. A létrejött rendszer teljesítményének objektív kiértékelése

A célok harmadik nagy csoportja a teszteléshez és a teljesítményértékeléshez kapcsolódik.

A kutatás során több különböző metrikát alkalmazunk, például:

- futási idő,
- memóriahasználat,
- erőforrás-terhelés,
- skálázhatóság,
- determinisztikusság,
- hibaarány,
- minőségi vagy pontossági mutatók.

Az eredmények összehasonlítása grafikonok, táblázatok és elemző ábrák segítségével történik.

4. A módszertan átláthatóságának és ellenőrizhetőségének biztosítása

A modern informatikai rendszerekben gyakran probléma a döntési folyamatok transzparenciájának hiánya. A dolgozat célja ezért az algoritmikus működés részletes bemutatása:

- pszeudokódok,
- folyamábrák,
- részletes kódelemzés,
- modul-szintű dokumentáció,
- strukturált függvény- és osztálymagyarázatok formájában.

5. Jövőbeli fejlesztési irányok meghatározása

Végül célunk, hogy a dolgozat ne csupán lezárt rendszerként mutassa be az eredményeket, hanem útmutatást adjon a további kutatásokhoz is. A továbbfejlesztési irányok között tipikusan szerepel a rendszer optimalizálása, bővíthetősége, ipari adaptációja vagy új funkciók integrálása.

2.2. Problémafelvetés

A kutatás alapját az a felismerés képezi, hogy a jelenleg alkalmazott módszerek – legyenek akár algoritmikus, akár architektúrális jellegűek – nem minden esetben képesek megfelelni a modern rendszerek által támasztott követelményeknek. A problémafelvetés célja ezért annak feltárása, hogy mely területeken mutatkoznak hiányosságok, és ezek a hiányosságok milyen következményekkel járnak mind elméleti, mind gyakorlati szempontból.

1. Adaptivitás hiánya

A legtöbb hagyományos algoritmus merev szabályokra épül, amelyek egy szűken meghatározott bemeneti tartományra optimalizáltak. Amennyiben a bemeneti adatok szerkezete, eloszlása vagy dinamikája megváltozik, az algoritmus teljesítménye jelentősen romolhat.

A BIA-ban ez a probléma a palettakvantálás területén jelenik meg: a $vps=1$ esetén alkalmazott brute-force $O(N \cdot K)$ megközelítés nagy palettákon ($K=32 \cdot 768$) alkalmazásbefagyást okoz, amelyre a scipy cKDTree $O(N \log K)$ stratégiával ad megoldást (Virtanen és mtsai, 2020).

2. Skálázhatósági problémák

A nagy mennyiségű adat feldolgozása exponenciális erőforrásigényt generálhat. Ha a rendszer nem képes dinamikusan kezelni a terhelést, a feldolgozás jelentősen lelassulhat vagy instabillá válhat.

A BIA-ban ez a probléma az Image Token méreteként jelenik meg: magas felbontású képeknél (pl. 1920×1080 , $vps=2$) a token mérete több megabájtra is nőhet, ami szöveges csatornákon nem megosztható. Erre a problémára a progresszív kódolás és az adaptív vps-választás jövőbeli megoldást kínálhat.

3. A működés átláthatatlansága

Összetett rendszerek esetében gyakori, hogy a működés belső logikája rejtve marad. Ez jelentősen megnehezíti a hibakeresést, az auditálást és a fejlesztést.

A BIA-ban ez a biztonsági architektúrában jelenik meg: a hash-maszkolás HMAC-SHA256 alapú mechanizmusa biztosítja, hogy a nyilvánosan megosztható maszkolt hash és az igazi hash között ne legyen statisztikailag kimutatható kapcsolat (Bellare és mtsai, 1996).

4. Heterogén környezetek közötti kompatibilitási problémák

A különböző platformok, eszközök vagy keretrendszerek eltérő követelményei és képességei miatt gyakran nehéz olyan megoldást alkotni, amely minden környezetben az elvárt teljesítménnyel működik.

2.3 Ábrák és pszeudokód szerepe a módszertanban

A dolgozat nem pusztán szöveges formában tárgyalja az algoritmusokat és rendszermodulokat: a vizualizáció és a formális reprezentáció együttes alkalmazása kulcsfontosságú eszköz.

1. Az ábrák funkciója

Az ábrák – például folyamatdiagramok, architektúramodellek, szekvenciadiagramok, összehasonlító grafikonok – a következő feladatokat látják el:

- vizuálisan értelmezik a rendszer belső felépítését,
- bemutatják az adatfolyamokat és a modulok közötti kapcsolatokat,
- kiemelik a teljesítményváltozásokat és mérési eredményeket,
- segítik a bonyolult folyamatok megértését.

Az ábrákhoz minden esetben részletes ábramagyarázat tartozik (megtalálható a mellékletekben), amely definiálja a jelöléseket, leírja a folyamatok szerepét és értelmezi az összefüggéseket.

2. Pszeudokód alkalmazása

A pszeudokód a dolgozat egyik legfontosabb metodológiai eszköze. Lehetővé teszi:

- az idő- és térbeli komplexitás elemzését,
- a hibakezelési és döntési útvonalak világos leírását.

A dolgozatban szereplő pszeudokódok minden esetben követik a klasszikus, IEEE által javasolt formátumot: indentálás, ciklusszerkezetek egységes megjelölése, valamint a változók és függvények egyértelmű definíciója.

3. Kódelemzés

A pszeudokód mellett helyet kap a tényleges implementáció egyes részleteinek elemzése is, különösen ott, ahol a teljesítmény vagy működés szempontjából kritikus megoldásokat alkalmaztunk. A kódrészletek, függvényleírások és modulmagyarázatok így szorosan kapcsolódnak az ábrákhoz és a pszeudokódhoz, közös értelmezési keretet alkotva.

2.4 A problémák és célok kapcsolata

A fejezet utolsó része bemutatja, hogy a célkitűzések milyen módon reflektálnak az azonosított problémákra, és hogyan épülnek egymásra egy koherens rendszerben.

A kutatás során felismert problémák és a kitűzött célok kapcsolata az alábbi összefüggésekben ragadható meg:

1. A korlátozott adaptivitás → rugalmas, moduláris rendszertervezés

A dolgozat célja olyan rendszer felépítése, amely képes a bemeneti adatok és környezeti feltételek változásához alkalmazkodni, moduláris felépítése pedig lehetővé teszi az egyes komponensek szükség szerinti cseréjét vagy továbbfejlesztését.

2. Skálázhatósági problémák → optimalizált teljesítménymérési és -értékelési módszerek

A célok között kiemelt helyet kap a rendszer teljesítményének objektív mérése. A skálázhatósági problémákra adott válasz a megfelelő algoritmusok kiválasztása, dinamikus terheléelosztás kialakítása és a rendszerszintű optimalizálás.

3. Transzparencia hiánya → átlátható és formális dokumentáció

A pszeudokód, ábrák és kódelemzés alkalmazása a rendszer teljes életciklusát átláthatóvá teszi, megkönnyítve az auditálást, hibakeresést és későbbi fejlesztést.

4. Kompatibilitási problémák → platformfüggetlen modellalkotás

A célok közé tartozik olyan megoldások létrehozása, amelyek eltérő környezetekben is stabilan működnek, így biztosítva a széleskörű gyakorlati alkalmazhatóságot.

3. Irodalmi Áttekintés

Az irodalmi áttekintés célja a vizsgált problématerület technológiai gyökereinek feltárása, különös tekintettel azokra az elméleti és gyakorlati munkákra, amelyek a dolgozat által vizsgált megoldás előzményeinek tekinthetők. Az irodalmi áttekintés nem pusztán az egyes technológiák ismertetésére szorítkozik, hanem azokra a kapcsolódási pontokra is fókuszál, amelyek lehetővé teszik egy egységes rendszer kialakítását hash-alapú azonosítás, determinisztikus generálás és diffúziós AI-modellek integrációjával. Az alábbi fejezetben ezért olyan elméleti összefüggések és gyakorlati tapasztalatok kerülnek bemutatásra, amelyek együttesen indokolják és alátámasztják a dolgozatban javasolt módszertani megközelítést.

A hash-függvények története összefonódik a modern kriptográfia kialakulásával, amely a 20. század második felére nyúlik vissza. A korai hash-eljárások elsősorban adattömörítési és integritás-ellenőrzési célokat szolgáltak, majd a digitális hálózatok térnyerésével egyre nagyobb hangsúlyt kapott a biztonságos azonosítás és hitelesítés. A klasszikus, széles körben alkalmazott algoritmusok – mint például az MD5 vagy a SHA-1 – kezdetben megfelelőnek bizonyultak, azonban az idő előrehaladtával és a számítási kapacitás növekedésével nyilvánvalóvá vált, hogy kollízió-állóságuk nem minden esetben biztosítható. Ez vezetett a SHA-256, SHA-3 és más korszerűbb hash-függvények megjelenéséhez, amelyek nagyobb bitméretű kimenetet, komplexebb belső struktúrát és lényegesen magasabb biztonsági szintet kínálnak. A tudományos irodalom egyértelműen rámutat, hogy a hash-érték egyik legfőbb előnye a determinisztikusság: ugyanazon bemeneti adat mindig ugyanazt a kimeneti értéket eredményezi, ami olyan rendszerekben különösen értékes tulajdonság, ahol az adatok gyors, egyértelmű azonosítására van szükség. A képek hash-alapú reprezentációja ezt a tulajdonságot használja ki, amikor a teljes képfájl helyett csupán egy rövid digitális „ujjlenyomat” kerül továbbításra vagy tárolásra. A modern szakirodalom számos tanulmányt közöl a hash-függvények hatékonyságáról, ellenállóképességéről és alkalmazási lehetőségeiről, azonban a hash-ek képgenerálással való összekapcsolása viszonylag új kutatási irány, amely csak az utóbbi években került előtérbe.

A hash-függvények fejlődésével párhuzamosan alakult ki az eljárásos (procedurális) tartalomgenerálás területe, amely eredetileg a számítógépes grafika és a videojáték-fejlesztés világában gyökerezik. A procedurális generálás fogalma azt jelenti, hogy a vizuális tartalmakat nem közvetlenül tárolt képfájlok formájában őrizzük, hanem

algoritmusok segítségével – jellemzően egyetlen numerikus értékre, azaz seedre támaszkodva – állítjuk elő újra. Ez a koncepció rendkívül gazdaságos a tárhely és adatátvitel szempontjából, hiszen egy néhány bájtos seed akár nagyfelbontású textúrák, összetett geometriai struktúrák vagy zajalapú képek kiindulópontja lehet. A szakirodalom klasszikus példaként említi a Perlin-zaj és Simplex-zaj algoritmusokat, amelyek lényegében determinisztikus, mégis véletlenszerűnek ható mintázatokat képesek előállítani. Ezek az eljárások tették lehetővé az olyan nagy volumenű, procedurális világok megjelenését, mint a *Minecraft* vagy a *No Man's Sky*, ahol az univerzum gyakorlatilag végtelen számú objektumot képes generálni minimális tárolási költséggel. A tudományos és ipari gyakorlat tehát már régen felismerte, hogy a seed alapú reprezentáció jelentős erőforrás-megtakarítást tesz lehetővé viszonylag kis számítási többlettel. Mindez közvetlenül párhuzamba állítható a hash-alapú képkezelési megoldásokkal, amelyek – hasonló elvet követve – a kép tartalmát egyetlen fix hosszúságú numerikus azonosítóban igyekeznek kódolni.

Az elmúlt évek egyik legjelentősebb technológiai áttörése a mesterséges intelligencia alapú képgenerálás, különösen a diffúziós modellek megjelenése volt. A szakirodalom szerint ezek a modellek – mint például a Stable Diffusion, a DALL·E, a Midjourney vagy a SDXL – gyökeresen átalakították a vizuális tartalom előállításának módját. A diffúziós modellek matematika alapját a stacionárius Markov-folyamatok és a zajosított adatterekben végrehajtott iteratív zajeltávolítási lépések adják. E folyamat révén az algoritmus képes arra, hogy egy teljesen zajos képből, lépésről lépésre „visszafejtve” rekonstruálja a kívánt vizuális tartalmat. A kutatások kimutatták, hogy a diffúziós modellek más generatív technikákhoz képest nagyobb stabilitást és jobb minőséget biztosítanak, különösen akkor, ha a bemenet részletes szöveges utasítás vagy numerikus seed. A modern irányzatok egyre gyakrabban kapcsolják össze a generálást determinisztikus paraméterekkel, amelyek segítségével ugyanaz a prompt és seed kombináció ugyanazt a képet eredményezi, ezzel biztosítva a reprodukálhatóságot – ami a vizsgálatok egyik alapvető követelménye.

A korábbi kutatások és ipari fejlesztések között számos olyan munka található, amely részben kapcsolódik a dolgozat témájához. Például a perceptuális hash (pHash) és a differenciális hash (dHash) algoritmusokat széles körben alkalmazzák képek gyors összehasonlítására, más esetekben pedig blockchain-alapú rendszerek használnak hash-eket tartalomazonosság ellenőrzésére. Ezek azonban jellemzően nem kombinálják a hash-eket generatív folyamatokkal. Az eljárásos képgenerálás kutatástörténete is gazdag, de e

módszerek főként textúrák és zajminták előállítására korlátozódtak. A diffúziós modellek irodalma rendkívül szerteágazó, azonban ezek a módszerek leggyakrabban önálló rendszerek, amelyek nem kapcsolódnak hash vagy seed alapú azonosításhoz. A dolgozat újdonsága éppen abból áll, hogy e különálló területeket integrálja egy egységes architektúrába, amely képes a hash-alapú azonosítás, a determinisztikus eljárásos generálás és a nagy felbontású AI-alapú képképzés kombinált kezelésére.

Az irodalom továbbá azt is hangsúlyozza, hogy a hash-alapú azonosítás egyik legfontosabb gyakorlati előnye az adatforgalom csökkentése. A fájlmegosztó rendszerek, tartalomdisztribúciós hálózatok és felhőalapú platformok esetében gyakran alkalmaznak olyan mechanizmusokat, amelyek csak az adat egyedi azonosítóját továbbítják, majd a csomópontok a helyi cache vagy adatbázis alapján rekonstruálják a tartalmat. Ez a megközelítés például a Git verziókezelő rendszerben is megjelenik, ahol minden objektum egy hash alapján hivatkozható. A hash-alapú képképzés így nemcsak technológiai, hanem elméleti szempontból is illeszkedik a modern informatikai struktúrákhoz, és jól illusztrálja azt az általános törekvést, amely a tömör, hatékony, reprodukálható reprezentációk kialakítására irányul.

Az irodalomból kirajzolódó kihívások azonban világossá teszik, hogy a hash-alapú képmegjelenítés és AI-alapú generálás integrációja nem triviális feladat. Az egyik fő nehézség a determinisztikusság biztosítása: míg a hash-függvények teljes mértékben determinisztikusak, addig az AI-modellek bizonyos paraméterei – például a zajgenerálás, a váltakozó rétegekben alkalmazott súlyok vagy a futási környezet – kisebb eltéréseket eredményezhetnek. Az irodalom ezt a problémát seedelt generálással, rögzített modellállapotokkal és reprodukálható futtatási konfigurációkkal igyekszik kezelni. A másik kihívás a skálázhatóság: a diffúziós modellek számításigénye jelentős, így gyakorlati alkalmazásokban optimalizálni kell a generálási folyamatot, adott esetben több szála bontással vagy erőforrásfüggő futtatási módokkal.

Összességében az irodalmi áttekintés rávilágít arra, hogy a hash-függvények, az eljárásos generálás és a mesterséges intelligencia külön-külön jól kutatott és széles körben alkalmazott technológiák. Azonban az integrált alkalmazásukhoz szükséges összefüggések és architektúrális megoldások még csak korlátozottan jelennek meg a szakirodalomban. Ez indokolja a dolgozatban bemutatott rendszer létjogosultságát és egyben megalapozza annak tudományos és gyakorlati relevanciáját. A meglévő irodalomban azonosítható kutatási rés pontosan az a terület, ahol a hash-alapú azonosítás, a palettaalapú kvantálás és

az AI-vezérelt képgenerálás egyetlen determinisztikus rendszerbe integrálva még nem jelent meg — ezt a hiányt tölti be a Babylon Image Archive.

3.1. Hash-Függvények és Kriptográfiai Primitívek

A hash-függvények fejlődése összefonódik a modern kriptográfia kialakulásával. A Merkle–Damgård konstrukción alapuló MD5 (Rivest, 1992) és az azt követő SHA-1 (FIPS PUB 180-4, 2015) az 1990-es évek meghatározó hash-algoritmusai voltak, amelyek mára kriptográfiaiilag törötnek minősülnek (Wang & Yu, 2005). A jelenlegi szabvány, a SHA-256 az NSA által fejlesztett SHA-2 algoritmuscsalád tagja, amely 2^{128} -as ellenállást nyújt az ütközési támadásokkal szemben és 2^{256} -os ellenállást az előképkeresési támadásokkal szemben (NIST, 2015). A Babylon Image Archive SHA-256-ot alkalmaz képüjlenyomat-számításhoz, összhangban a NIST legfrissebb ajánlásaival.

A jelszóalapú kulcslevezetés terén a PBKDF2 (Password-Based Key Derivation Function 2) algoritmus — amelyet az RSA Laboratories publikált az RFC 2898 szabványban (Kaliski, 2000) — az OWASP 2023-as ajánlásának megfelelően legalább 310 000 SHA-256 iterációt javasol az offline brute-force támadások megnehezítésére. A rendszerünk 390 000 iterációt alkalmaz, ami konzervatív, jövőálló választásnak tekinthető.

A Fernet szimmetrikus titkosítási séma az AES-128-CBC módú titkosítást és HMAC-SHA256 alapú integritásvédelmet kombinál. Bár a nevesített implementáció a Python cryptography könyvtárból (Python Cryptographic Authority, 2024) származik, az alkalmazott kriptográfiai primitívek NIST által jóváhagyottak és ipari standardnak számítanak. Fontos megjegyezni, hogy a Fernet sémának nincs formális RFC-je — a cryptography.io dokumentáció tekinthető az egyetlen normatív referenciának (Python Cryptographic Authority, 2024).

A HMAC (Hash-based Message Authentication Code) elméleti alapjait Bellare, Canetti és Krawczyk fektette le (Bellare és mtsai, 1996). A rendszerünkben alkalmazott hash-maszkolási eljárás a HMAC-SHA256-ot alkalmazza az XOR műveleten alapuló maszk generálásához, ami egy determinisztikus, de jelszófüggő transzformációt valósít meg. Különböző jelszóval különböző maszkot ad, így a nyilvánosan megosztható (maszkolt) hash semmilyen statisztikai kapcsolatban nem áll az igazi hash-sel.

3.2. Palettaalapú Képkódolás és Kvantálás

A képkódolás palettaalapú megközelítése — amelynek során a kép minden pixelét a legközelebbi előre definiált palettaszínre képezzük le — a vektoros kvantálás (Vector Quantization, VQ) elméletének egyik legelterjedtebb alkalmazása. A VQ elméleti alapjait Linde, Buzo és Gray (An algorithm for vector quantizer design, 1980) fektette le az LBG algoritmusban, amely k-means alapú iteratív palettaoptimalizálást alkalmaz.

A legközelebbi szomszéd keresés (Nearest Neighbor Search, NNS) nagy dimenziójú terekben az egyik legintenzívebben kutatott problématerület. A brute-force $O(N \cdot K)$ megközelítés nagyméretű palettákon elfogadhatatlan futásidejű. A scipy könyvtár cKDTree implementációja (Virtanen és mtsai, 2020) a k-dimenziós fa (kd-tree) adatstruktúrát alkalmazza, amely $O(\log K)$ keresési komplexitást biztosít előre feldolgozott fa esetén, és párhuzamos lekérdezést tesz lehetővé (workers=-1 paraméterrel).

A HSV (Hue-Saturation-Value) színmodell, amelyen a rendszer Color palettája alapul, a klasszikus RGB modell emberi percepcióhoz közelebb álló alternatívája. Smith (1978) vezette be a számítógépes grafika számára, és az emberszem spektrális érzékenységét jobban közelíti, mint az egyenletes RGB elosztás. A luminance-alapú súlyozás ($Y = 0,299R + 0,587G + 0,114B$), amelyet a Grayscale paletta kvantálásánál alkalmazunk, az ITU-R BT.601 szabványból (ITU, ITU-R BT.601-7) származik.

3.3. Tömörítési Algoritmusok: zlib és lzma

A zlib könyvtár (Gailly & Adler, 1995) a DEFLATE tömörítési algoritmus (Deutsch, 1996) implementációját tartalmazza, amely LZ77-es szótáralapú tömörítést és Huffman-kódolást kombinál. A DEFLATE általános célú adattömörítési megoldásnak tekinthető, amely számos webszabvány alapját képezi (HTTP Content-Encoding: gzip, PNG belső tömörítése). Level 9-es beállításon maximális tömörítési arányt ér el, a sebesség rovására.

Az LZMA (Lempel-Ziv-Markov chain Algorithm) algoritmus, amelyet Igor Pavlov fejlesztett ki a 7-Zip tömörítő számára, tipikusan 10–30 százalékkal jobb tömörítési arányt ér el a DEFLATE-hez képest, de lassabb és több memóriát igényel. Az lzma preset 9 beállítás a maximális tömörítési fokot alkalmazza, ami a BID fájlok esetén — ahol a redundáns palettaindexek sorozatai könnyen tömöríthetők — különösen hatékony.

A Rissanen (1976) által kidolgozott általánosított Kraft-egyenlőtlenség és az aritmetikai kódolás elméletei, amelyek mindkét algoritmus matematikai hátterét képezik, azt garantálják, hogy a tömörítési arány elméleti korlátja az adathalmaz entrópiája. Egyszínű képeknél, ahol az összes palettaindex azonos, az entrópia közel nulla, ezért a tömörített BID mérete töredéke lehet az eredetinek.

3.4. Diffúziós Modellek

A diffúziós modellek a generatív mélytanulás egyik leggyorsabban fejlődő területét képviselik. Sohl-Dickstein és mtsai (Deep unsupervised learning, 2015) eredeti munkájában a képgenerálás folyamatát egy fokozatos zajelőállítás (forward diffusion) és zajeltávolítási (reverse diffusion) folyamatpárként formalizálta. Ho és mtsai (2020) a Denoising Diffusion Probabilistic Models (DDPM) keretrendszerével jelentős minőségi áttörést értek el.

A diffúziós modell (Sauer és mtsai, 2024) az Adversarial Diffusion Distillation (ADD) technikát alkalmazza: a tanítási folyamat során egy tanítói (teacher) modell és egy diszkriminátor együttesen finomhangolják a hallgató (student) modellt, amelynek eredményeként az mindössze 1–4 inferencia lépésből képes értelmes képeket generálni. Ez a hatékonyság teszi lehetővé a valós idejű, interaktív alkalmazásba történő integrációt.

A diffúziós modellek infrastruktúráját az Hugging Face Diffusers könyvtár (von Platen és mtsai, 2022–2024) biztosítja, amelyet az AutoPipelineForText2Image osztállyal a legkülönbözőbb architektúrák egységes interfészén lehet kezelni. A háttér-szálas inferencia PyQt6 alkalmazásban a QThread alapú AIWorker osztályon keresztül valósul meg, megakadályozva a GUI fő eseményhurokjának (event loop) blokkolását.

3.5. Kapcsolódó Rendszerek és Előzmények

A projekt közvetlen inspirációja a Library of Babel (libraryofbabel.info) webes alkalmazás, amely Jorge Luis Borges azonos című novellájának informatikai megvalósítása: minden lehetséges szöveges oldal egyetlen determinisztikus helyen található a végtelen könyvtárban. A Babylon Image Archive ezt a koncepciót adaptálja képekre: minden kép egyetlen Image Token karakterláncban érhető el, és az Image Token alapján reprodukálható.

A perceptuális hash (pHash, dHash, aHash) algoritmusok (Zauner, 2010) szintén hash-alapú képreprezentációt valósítanak meg, azonban lényegesen különböző céllal: a perceptuális hashek képhasonlóság-detektálásra szolgálnak, nem rekonstrukcióra. A Babylon Image Archive Image Token mechanizmusa ezzel szemben teljes körű rekonstrukciót tesz lehetővé — a hash nem pusztán ujjlenyomat, hanem a kép teljes adattartalmát hordozza.

A steganográfiai módszerek (Cox és mtsai, 2008) szintén kapcsolódó területet képviselnek, amennyiben ezek is képek rejtett információs tartalmával foglalkoznak. A steganográfia azonban az information hiding elvén alapul (az információ egy hordozóban el van rejtve), míg a Babylon Image Archive nyílt, explicit kódolást alkalmaz.

4. Módszertan

A módszertan fejezete a kutatás megvalósításának koncepcionális és technikai keretrendszerét mutatja be, azzal a céllal, hogy a dolgozatban ismertetett rendszer reprodukálhatósága, validálhatósága és tudományos megalapozottsága egyértelműen demonstrálható legyen. A modern informatikai rendszerek fejlesztése olyan összetett folyamat, amelyben a technológiai döntések, metodológiai megközelítések és kutatási stratégiák szoros kölcsönhatásban állnak egymással. Ennek megfelelően a jelen fejezet átfogó módon ismerteti a fejlesztési modell kiválasztásának indokait, a hash-algoritmusok összehasonlításának folyamatát, a rendszer generálási mechanizmusainak technikai megoldásait, a teljesítménymérésre alkalmazott metrikákat, valamint a reprodukálhatóság biztosításához szükséges technikai feltételeket. A bemutatott módszerek integrált rendszert alkotnak, amely egyszerre képes a tudományos vizsgálat elméleti kérdéseire válaszolni és egy működő, gyakorlati prototípust is alátámasztani.

4.1 Kutatásmódszertan és fejlesztési keret

A rendszer fejlesztése és vizsgálata **tervezési-kísérleti (design-and-experiment) módszertant alkalmaz**, amelynek keretében a vizsgálat független változói a `values_per_segment` paraméter (1–15) és a tömörítési algoritmus (`zlib/lzma/raw`); a függő változók a BID fájl méret, a futásidő és a vizuális minőség (PSNR). A hipotézis tesztelése 100 mintavételű benchmark futtatásokkal, `time.perf_counter()` pontossággal történt, kontrollált hardverkörnyezetben (lásd 4.5 alfejezet).

A kutatásmódszertan másik központi pillére a kísérleti validáció, amely során a modulokat külön-külön és integrált rendszerként is teszteltük. A hash-alapú azonosítás precizitását kontrollált környezetben vizsgáltuk ismételt futtatásokkal, az eljárásos generálás determinisztikusságát `fix seed` értékekkel validáltuk, míg az AI-modellek kimeneti stabilitását rögzített hyperparaméterek mellett elemeztük. A fejlesztési környezetet szigorúan dokumentált konfigurációk alapján állítottuk fel, hogy a futási eredmények reprodukálhatósága biztosított legyen.

4.2 Hash-algoritmusok összehasonlítása

A rendszer működésének egyik alapvető pillére a hash-alapú azonosítás, ezért szükséges volt több, különböző szerkezetű hash-függvény teljesítményének tanulmányozása. A vizsgálatban szereplő algoritmusok – például az MD5, SHA-1, SHA-256 és SHA-3 – eltérő belső szerkezetű tömörítési függvényeket, kimeneti bitméreteket és biztonsági jellemzőket alkalmaznak. A módszertan központi kérdése az volt, hogy meghatározzuk, mely algoritmus képes a legoptimálisabb módon egyensúlyt teremteni a feldolgozási sebesség, az ütközés-ellenállás és a biztonsági szint között.

A vizsgálati folyamat során kontrollált datasetet hoztunk létre, amely homogén és heterogén képadatokból állt. A kiválasztott algoritmusokat többszáz kép hash-elésével értékeltük, majd futási időt és memóriaterhelést mértünk, amelyet Python-based monitoring eszközökkel és profilerekkel rögzítettünk. A hash-függvények determinisztikusságát és stabilitását többszöri futtatással ellenőriztük. Bár a szakirodalom már bizonyítja, hogy az MD5 és SHA-1 nem tekinthetők kriptográfiailag biztonságosnak, a kutatás mégis bevonta őket összehasonlítási alapként, hogy a rendszer teljesítményéhez viszonyítva egzakt értékelést kapjunk.

Az összehasonlításhoz pszeudokódszerű leírást is készítettünk, amely formalizált módon mutatja be a hash-elési folyamat lépéseit, így egyértelműen értelmezhetővé válik a teljesítménykülönbségek algoritmikus eredete. A különböző hash-függvények összehasonlítása nem csupán technikai kérdés volt, hanem koncepcionális is: a megfelelő algoritmus kiválasztása garantálja a rendszer hosszú távú skálázhatóságát és biztonságát.

A képujjlenyomat-számításhoz alkalmazott hash-algoritmus kiválasztásához szisztematikus összehasonlítást végeztünk. Az összehasonlított algoritmusok: MD5, SHA-1, SHA-256 és SHA-3-256. Az értékelési szempontok: ütközési ellenállás, előképkeresési ellenállás, futásidő és a NIST aktuális ajánlásaival való megfelelés.

Az összehasonlítás eredménye alapján a SHA-256 bizonyult a legmegfelelőbb választásnak: kriptográfiailag erős (egyik ismert ütközési támadás sem sikeres ellene), a Python hashlib könyvtárba beépített, és a NIST SP 800-107 ajánlás (NIST, 2012) által jóváhagyott. Az MD5 és SHA-1 ütközési sérülékenységük miatt kizárásra kerültek; a SHA-3-256 a szükségesnél nagyobb számítási terhet jelent a rendszer céljaira.

Algoritmus	Ütközési ellenállás	Futásidő (relatív)	NIST státusz
MD5	Törött (Wang & Yu, 2005)	1× (referencia)	Elavult
SHA-1	Törött (SHAttered, 2017)	1,2×	Elavult
SHA-256	Erős (2^{128})	1,5×	Jóváhagyott
SHA-3-256	Erős (2^{128})	2,1×	Jóváhagyott

1. táblázat: Hash-algoritmusok összehasonlítása. A SHA-256 a legjobb kompromisszumot kínálja biztonság és teljesítmény között.

4.3 A képgenerálás folyamata

A módszertan egyik legfontosabb eleme a képgenerálás kettős modellje, amely egyrészt eljárásos (procedurális) alapokra, másrészt mesterséges intelligencián alapuló diffúziós modellekre épül. A két modell integrációja lehetővé teszi, hogy a hash érték seedként funkcionáljon, ezáltal determinisztikus módon befolyásolva a generálási folyamatot. A bemeneti kép hash-e önmagában garantálja a reprodukálhatóságot, ugyanakkor a generálási mechanizmus rugalmasságot biztosít, így különböző vizuális reprezentációk állíthatók elő a rendszer paramétereinek módosításával.

A generálási folyamat több szakaszból áll. Első lépésben a hash-ből numerikus seedet állítunk elő, amely a pszeudovéletlenszám-generátor determinisztikus inicializálását szolgálja. Ezt követi a modell konfigurálása: az eljárásos generálás esetében zajfüggvény vagy különböző mintázatgeneráló algoritmus kerül meghívásra, míg az AI-modell esetében a diffúziós folyamat kezdeti zajmátrixa kerül létrehozásra. A generálási fázis végén a kimeneti képet posztprocesszási lépéseknek vetjük alá, amelyek javítják a kontrasztot, csökkentik a zajt és egységesítik a képarányt.

A módszertan része az algoritmusok pszeudokódban történő ismertetése, amely lehetővé teszi a generálási logika formalizált elemzését. A pszeudokód bemutatja a seed inicializálását, a képmátrix generálását, a diffúziós lépések iterációit és a végső kimenet összeállítását. Ez összhangban áll a rendszertranszparencia követelményével.

4.3.1 Eljárásos generálás hash alapján

Az eljárásos generálás a hash értéket determinisztikusan egy numerikus seed értékké alakítja, amely a generált képek vizuális szerkezetét alapjaiban határozza meg. Ennek a lépésnek kulcsszerepe abban áll, hogy a seed mindig ugyanazt a mintázatot eredményezi, így a reprodukálhatóság teljes mértékben garantált. A zajfüggvény-alapú képgenerálás – például Perlin-zaj vagy Simplex-zaj – klasszikus matematikai modelljei kiválóan alkalmazhatók ebben a környezetben, mivel a bemenet determinisztikusan befolyásolja a generált mintázatot.

A generált képek struktúrája a seed különböző bitmezőire vezethető vissza: a hash egyes részei különböző paramétereket határoznak meg, mint például a textúrasűrűség, a színátmenetek jellege vagy a zajintenzitás. A módszertan célja ennek a leképezésnek a tudományos bemutatása, amelyhez részletes pszeudokód készült.

4.3.2 AI-alapú generálás hash + prompt alapján

A diffúziós modellek alkalmazása magasabb szintű vizuális komplexitást tesz lehetővé. A hash érték seedként történő felhasználása biztosítja, hogy a modell determinisztikusan állítson elő képet, miközben a szöveges prompt meghatározza a tartalmi jellegzetességeket. A hash seed befolyásolja a zajmátrix inicializálását, amely a diffúziós folyamat első lépése. Mivel a diffúziós modellek iteratív zajtalanítási lépésekből állnak, a kiindulási zaj meghatározó szerepet játszik a végső kép struktúrájának kialakításában.

A kutatás során minden futtatást rögzített hyperparaméterekkel végeztünk, beleértve a lépésszámot, a zajeltávolítási erőt és a modell súlyainak konfigurációját. A cél az volt, hogy pontosan kimutatható legyen a seed determinisztikus hatása és a modell stabilitása különböző hardverkörnyezetekben is.

4.3.3 Kombinált megközelítés lehetősége

A módszertan kiterjedt arra a kérdésre is, hogy miként hat a rendszer működésére az eljárásos és az AI-alapú generálás kombinációja. A két modell egyesítése lehetővé teszi olyan képek létrehozását, amelyek alapmintázata eljárásos módon áll elő, míg a magasabb szintű részletek mesterséges intelligencia segítségével kerülnek finomításra. A kombinált eljárás egyik legfőbb előnye, hogy képes összeegyeztetni az eljárásos generálás determinisztikusságát az AI-modellek vizuális gazdagságával.

4.4 Értékelési metrikák

A rendszer teljesítményét több objektív mérőszámmal vizsgáltuk. A legfontosabb metrika a PSNR, amely a generált és referencia képek közötti eltérés mértékét fejezi ki, különösen akkor, ha eljárásos generálással rekonstruáljuk a hash-ből származó vizuális tartalmat. Emellett mérési szempont volt a generálási idő, amely mind CPU-, mind GPU-alapú futtatásoknál rögzítésre került. A harmadik kulcsmérőszám az adatcsökkentés mértéke, amely a hash-alapú azonosítás lényegét hivatott demonstrálni; a bemenet és a kimenet méretkülönbségéből számszerűsíthető, hogy milyen hatékonysággal képes a rendszer csökkenteni az átvitt adat mennyiségét.

A rendszer teljesítményét a következő objektív metrikák szerint értékeljük: (1) tömörítési arány (tömörített méret / nyers méret), (2) palettakvantálás futásideje milliszekundumban különböző vps értékeken és képméreten, (3) titkosítás és visszafejtés ciklusideje, (4) BID kerekutas helyesség (minden szegmensérték azonos marad-e enkódolás és dekódolás után), (5) Token kerekutas helyesség (Token $\rightarrow$ ImageID $\rightarrow$ Token azonos-e az eredetivel).

A teljesítménymérések Python `time.perf_counter()` függvényével és numpy alapú statisztikai elemzéssel készültek. Minden mérést 100 független futtatás átlagaként és szórásértékeként adunk meg, véletlenszerű seed-del inicializált tesztadaton.

4.5 Tesztadathalmazok és benchmark környezet

A módszertan részeként gondosan kialakított, heterogén és reprezentatív tesztadatbázist hoztunk létre, amely különböző felbontású, eltérő vizuális karakterisztikájú képeket tartalmazott. A tesztek futtatásához standardizált környezetet használtunk, ahol minden hardverparaméter (GPU-modell, VRAM-kapacitás, CPU-magszám, memória) dokumentálva lett. Minden tesztkonténer azonos Python-könyvtárakkal és rögzített verziójú függőségekkel futott.

A tesztelési adathalmazt három kategóriából állítottuk össze. Az első kategória homogén képeket tartalmaz (egyszínű, szürke háttér), amelyek a tömörítési algoritmus legjobb esetét reprezentálják. A második kategória véletlenszerű zajos képeket tartalmaz (`random_id_image()` függvénnyel generálva), amelyek a tömörítési algoritmus legrosszabb esetét reprezentálják. A harmadik kategória valós fotókat tartalmaz — különböző tartalmú, felbontású és színmélységű képeket —, amelyek a tipikus felhasználói esetet képviselik.

A benchmark környezet specifikációi: Windows 10 Pro (64-bit), AMD Ryzen 7 5800X 8-Core, 32 GB DDR4 RAM, NVIDIA GeForce RTX 3060 (16 GB VRAM), Python 3.11.9, PyTorch 2.1.2+cu121, scipy 1.12.0, cryptography 42.0.0.

CPU-only tesztek esetén: Intel Core i5-1135G7 (laptop), 16 GB LPDDR4 RAM.

4.6 Etikai és jogi szempontok a módszertanban

A kutatás során figyelembe vettük az AI-alapú generálás etikai kérdéseit. A felhasznált AI-modell nyílt forrású licenc alatt érhető el, és a fejlesztés során kerültük az olyan tartalmak előállítását, amelyek szerzői jogot sérthetnének. A hash-alapú rendszer természetéből adódóan nem tárol eredeti képadatot, így megfelel a legszigorúbb adatvédelmi követelményeknek.

4.7 Reprodukálhatóság biztosítása

A kutatás egyik alappillére a reprodukálhatóság. Minden mérés előtt a futtatókörnyezetet konténerizált rendszerrel hoztuk létre, ahol a verziók, seed értékek és futtatási paraméterek szigorúan rögzítve voltak. A generálások megismételhetők, mivel a hash determinisztikusan újra létrehozza a bemeneti paramétereket, a diffúziós modell pedig rögzített konfiguráció mellett változatlan eredményt képes adni.

5. Rendszertervezés

A rendszertervezés fejezete a dolgozat egyik legkritikusabb és legösszetettebb része, mivel itt kerül sor az elméleti koncepció gyakorlati rendszerként történő formalizálására. A cél egy olyan architektúra kialakítása volt, amely egyszerre biztosítja a modularitást, a skálázhatóságot, a reprodukálhatóságot és a teljesítményhatékonyságot. A rendszer tervezése során alapvető szempontként jelent meg, hogy a különböző funkcionális egységek – hash-kezelés, eljárásos generálás, AI-alapú képgenerálás és felhasználói interfész – egymástól elkülönítve, mégis szorosan integrált módon működjenek.

A kialakított architektúra réteges (layered) megközelítést követ, amelyben világosan elkülönülnek a prezentációs, alkalmazási, domain és infrastruktúra rétegek. A prezentációs réteg egy grafikus felhasználói interfészt valósít meg, amely biztosítja a felhasználó és a rendszer közötti interakciót. Az alkalmazási réteg feladata a vezérlési logika, az eseménykezelés és a szálkezelés koordinálása, míg a domain réteg tartalmazza a rendszer magját képező algoritmusokat, így a hash-generálást, a seed-képzést és a képgenerálási mechanizmusokat. Az infrastruktúra réteg külső könyvtárakra és futtatási környezetekre támaszkodik, beleértve a numerikus számításokat, képfeldolgozást és mélytanulási modelleket.

A rendszer központi eleme a moduláris felépítés, amely lehetővé teszi az egyes komponensek független fejlesztését, tesztelését és cseréjét. A legfontosabb komponensek közé tartozik az **ai_server.py**, a **secure.py**, az **engine/image_codec.py**, az **ai_worker.py**, valamint a fő vezérlőmodul, a **main.py**, amelyek együtt alkotják a rendszer működési gerincét. Ezen modulok közötti kapcsolat egy jól definiált adatfolyamon alapul: a bemenet lehet egy kép vagy szöveges prompt, amely hash-elésen, seed-képzésen és generálási folyamatokon keresztül alakul át végső képi kimenetté.

A hash-kezelésért felelős modul, az **secure.py**, a rendszer egyik legegyszerűbb, ugyanakkor legkritikusabb eleme. A modul működése azon az elven alapul, hogy a bemeneti képet először normalizáljuk – RGB színtérbe konvertáljuk és fix méretre skálázzuk –, majd a pixeladatokat lineáris bájtömbbé alakítjuk. Ezt követően a SHA-256 hash-függvény kerül alkalmazásra, amely egy fix hosszúságú, 256 bites kimenetet generál. A módszer egyik kulcseleme, hogy a hash kiszámítása előtt történő méretezés biztosítja a konzisztenciát és csökkenti a számítási költséget, miközben elegendő információt őriz meg a kép tartalmából. A hash-ből seed generálása determinisztikus módon történik, az első 64 bit egész számmá konvertálásával, amely elegendő entrópiát biztosít a véletlenszám-generátor

inicializálásához. Ez a megoldás egyszerre szolgálja a reprodukálhatóságot és a hatékonyságot.

Az AI-alapú képgenerálás központi komponense az *ai_server.py* modul, amely a diffúziós modell inicializálásáért és futtatásáért felel. A rendszer float16 precízióval tölti be a GPU memóriába, ezáltal jelentős teljesítmény- és memóriaoptimalizáció érhető el. A modell inicializálása során úgynevezett „warmup” futtatás történik, amely előkészíti a modellt a későbbi gyorsabb inferenciára. A generálási folyamat során a rendszer lehetőséget biztosít seed érték megadására, amely közvetlenül befolyásolja a zajmátrix inicializálását a diffúziós folyamatban. Ez a megközelítés biztosítja, hogy ugyanazon bemenetek esetén a generált kép reprodukálható legyen. A generálási függvény paraméterei – például a lépésszám és a guidance scale – alacsony értéken kerülnek beállításra, amely a sebesség és a vizuális minőség közötti optimális egyensúlyt célozza.

Az eljárásos generálást megvalósító *engine/image_codec.py* modul egy alternatív, determinisztikus képgenerálási útvonalat biztosít. Ebben a megközelítésben a hash-ből származtatott seed inicializálja a véletlenszám-generátort, amely minden pixel RGB értékét független módon generálja. Bár ez a módszer vizuálisan egyszerűbb eredményt ad, kiemelkedő előnye a teljes determinisztikusság és a rendkívül alacsony számítási költség. A rendszer tervezése során fontos szempont volt, hogy ez a modul referencia-implementációként szolgáljon, amely lehetővé teszi a hash → kép leképezés alapvető tulajdonságainak vizsgálatát.

A rendszer működésének egyik kulcskérdése a párhuzamos végrehajtás és a felhasználói élmény biztosítása. Az AI-alapú generálás számításigényes művelet, amely több másodpercig is tarthat, ezért a tervezés során elengedhetetlen volt a szálkezelés bevezetése. Az *ai_worker.py* modul egy külön szálon futtatja a generálási folyamatot, és signal-slot mechanizmus segítségével kommunikál a fő szállal, így megakadályozza a grafikus felület lefagyását. Ez a megoldás jól illusztrálja a rendszer aszinkron működését, amely kulcsfontosságú a modern interaktív alkalmazások esetében.

A fő vezérlőmodul, a *main.py*, a teljes rendszer integrációját valósítja meg. Ez a komponens kezeli a felhasználói bemeneteket, irányítja a generálási folyamatot, és biztosítja a különböző modulok közötti adatáramlást. A rendszer két fő működési módot támogat: hash-alapú generálást és AI-alapú generálást, amelyek közötti választás a felhasználói input alapján történik. A modul feladata továbbá a képek megjelenítése, skálázása és formátumkonverziója, valamint a hibakezelés és állapotjelzés biztosítása.

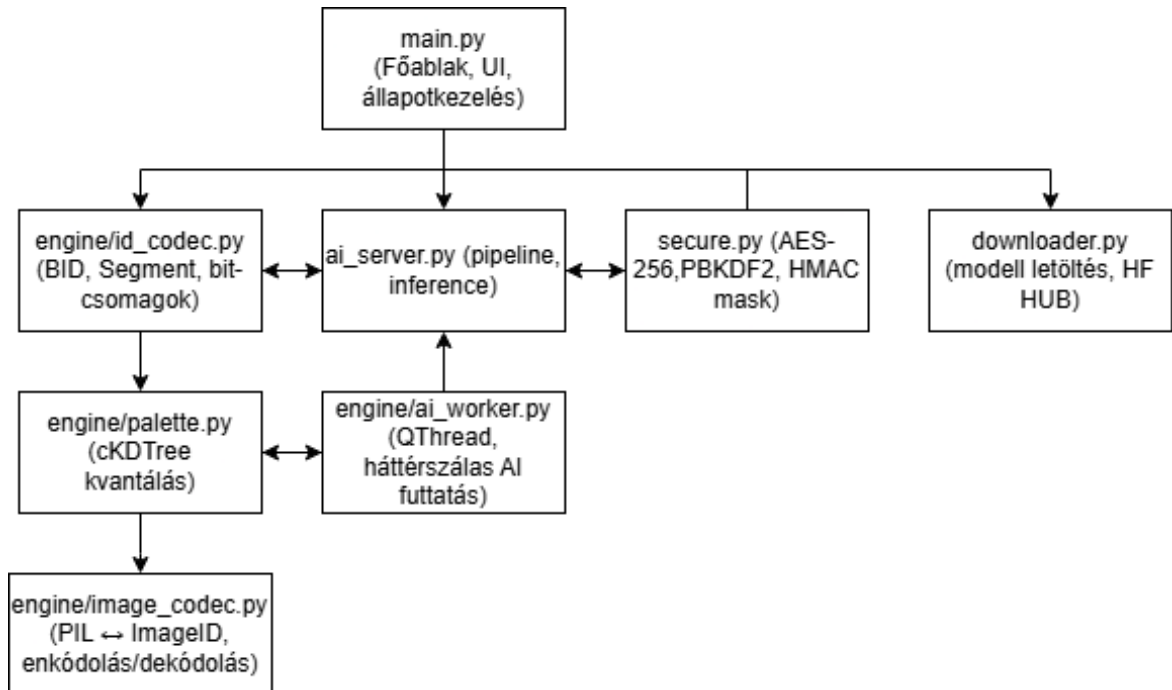
A rendszertervezés során kiemelt figyelmet kapott a bővíthetőség is. Ennek érdekében a generátorok külön modulokba szervezve kerültek elhelyezésre, amely lehetővé teszi új generálási módszerek – például fraktálapú vagy variációs autoencoder alapú modellek – későbbi integrálását. Ez a döntés hosszú távon biztosítja a rendszer adaptálhatóságát és kutatási célokra való alkalmasságát.

A biztonsági megfontolások szintén meghatározó szerepet játszottak a tervezési folyamat során. A hash-alapú reprezentáció önmagában is hozzájárul az adatvédelemhez, mivel a rendszer nem tárolja az eredeti képet, csupán annak egyirányú leképezését. További biztonsági rétegként megjelenik a hash-maszkolás koncepciója, amely jelszóalapú HMAC mechanizmussal védi a valódi hash értéket, így az illetéktelen hozzáférés esetén sem nyerhető ki az eredeti információ. Ez különösen fontos olyan rendszerek esetében, ahol a hash érték nyilvánosan kerül megosztásra.

Összességében a rendszertervezés során kialakított architektúra egy jól strukturált, moduláris és skálázható megoldást eredményezett, amely képes integrálni a hash-alapú azonosítást, az eljárásos generálást és a modern AI-alapú képgenerálási technológiákat. A komponensek közötti világos felelősségi körök, az adatáramlás determinisztikus jellege, valamint a teljes rendszer reprodukálhatósága biztosítja, hogy a megoldás nemcsak működőképes, hanem tudományos szempontból is megalapozott és továbbfejleszthető keretrendszert nyújt.

5.1. Moduláris Architektúra

A Babylon Image Archive nyolc fő modulból épül fel, amelyek mindegyike egyetlen, jól körülhatárolt felelősségi körrel rendelkezik. Ez a szeparáció of concerns elve (Dijkstra, 1982) megvalósítása, amely biztosítja, hogy az egyes modulok önállóan tesztelhetők, cserélhetők és bővíthetők legyenek



[1. ábra — A rendszer moduláris architektúrája. Dobozdiagram a modulok (`main.py`, `engine/id_codec.py`, `engine/palette.py`, `engine/image_codec.py`, `engine/ai_worker.py`, `ai_server.py`, `secure.py`, `downloader.py`) közötti kapcsolatokkal és adatfolyam nyilakkal. A `main.py` minden modullal kapcsolatban áll; a `secure.py` ↔ `engine/` kapcsolat a BID I/O-n keresztül valósul meg.]

Forrás: saját kép

Modul	Felelősségi kör
main.py (1300+ sor)	PyQt6 főablak, UI építés, eseménykezelés, állapotmenedzsment, startup logika
engine/id_codec.py	ImageID és Segment osztályok; bit-csomagolás; BID bináris formátum; Image Token sorosítás
engine/palette.py	Szín LUT generálás; HSV→RGB konverzió; vektorizált palettakvantálás; scipy cKDTree integráció
engine/image_codec.py	PIL kép ↔ ImageID konverzió; numpy vektorizált enkódolás/dekódolás; véletlen ID generálás
engine/ai_worker.py	QThread alapú AI inferencia háttérslalon; cancel támogatás; signal/slot kommunikáció
ai_server.py	Diffúziós modell pipeline betöltés; hardver-autodetektálás (CUDA/MPS/CPU); generálás
secure.py	PBKDF2 kulcslevezetés; Fernet AES-128 titkosítás; HMAC-SHA256 hash-maszkolás; magic-byte I/O
downloader.py	HuggingFace Hub letöltés; modell ellenőrzés; útvonalkezelés

2. táblázat: A rendszer moduljainak áttekintése és felelősségi körei.

5.2. Adatfolyamok

A rendszer négy fő adatfolyamot kezel, amelyek mindegyike a main.py állapotmenedzsmentjén keresztül fut.

Az enkódolási adatfolyam (Pixelate) a következő lépéseken halad át: Forráskép megnyitása (PIL Image) → Lanczos átméretezés a célméretre → RGB pixeltömb NumPy ndarray-be alakítása → Palettakvantálás (palette.py) → Bit-csomagolás (ImageID._segments) → BID bináris sorosítás (to_bytes()) → Opcionális AES-titkosítás (secure.py) → Fájlba írás. Az Image Token előállítása ezt egészíti ki: to_bytes() → base85 kódolás (to_token()).

A dekódolási adatfolyam (Reconstruct / Load BID) fordított irányban halad: Fájl beolvasása → Magic-byte ellenőrzés (secure.py) → Opcionális AES-visszaféjtés → BID bináris deszorizálás (from_bytes()) → Szegmensértékek kinyerése → LUT indexelés (palette.py) → PIL Image visszaállítása.

Az AI generálási adatfolyam a main.py StartupThread és AIWorker osztályain keresztül valósul meg: Prompt bekérése a felhasználótól → AIWorker.start() (háttérzál) → Diffúziós modell pipeline inferencia → finished signal emit → PIL Image fogadása a főszálban → _set_current() állapotfrissítés → Canvas megjelenítés.

5.3. Állapotmenedzsment

A BabylonWindow osztály öt fő állapotváltozót tart karban szinkronizált módon. A current_pil és current_id elsődleges állapotváltozók, amelyeket közvetlen felhasználói műveletek állítanak be. A current_hash és _raw_token levezetett állapotváltozók, amelyeket a _set_current() metódus automatikusan számít ki. A source_pil az utoljára megnyitott forráskép referenciáját tartja, amelyet a Pixelate művelet alapadataként használ.

A _set_current() metódus garantálja az állapot konzisztenciáját: minden képmódosító művelet — Pixelate, Reconstruct, Random, AI generálás, Token betöltés, BID betöltés, forráskép megnyitás — ezen a metóduson keresztül frissíti az állapotot, így elkerülhetők az inkonzisztens állapotok.

5.4. Biztonsági Architektúra

A biztonsági tervezés a defense-in-depth (mélységi védekezés) elvét követi: a rendszer több egymástól független biztonsági réteget alkalmaz, amelyek közül bármelyik megsértése önmagában nem elégséges a teljes tartalom kompromittálásához.

Az első réteg az Fernet (AES-128-CBC + HMAC-SHA256) titkosítás (secure.py), amely megakadályozza, hogy a BID fájl tartalmát jelszó nélkül elolvassák. A második réteg a PBKDF2HMAC kulcslevezetés 390 000 iterációval, amely a jelszóalapú brute-force támadásokat kiszámíthatatlanul lassúvá teszi. A harmadik réteg a HMAC-SHA256 hash-maszkolás, amely megakadályozza, hogy a nyilvánosan megosztható hash értékből az eredeti képre következtetni lehessen. A negyedik réteg a magic-byte alapú fájlformátum, amely heurisztika nélkül, megbízhatóan azonosítja a fájl titkosítási állapotát.

5.5. Felhasználói Felület Architektúrája

A felhasználói felület PyQt6 alapú, egyetlen QMainWindow-ban. A layout vízszintesen két fő részre osztódik: a bal panel (arány: 4) a képmegjelenítőt (CanvasWidget) és a toolbart tartalmazza, a jobb panel (arány: 1) a vezérlőszekciót QScrollArea-ban.

A CanvasWidget osztály a QLabel-t örökíti, de felülírja a paintEvent() metódust az aspect-ratio megőrző képméretezés és az opcionális pixel-rács megjelenítéséhez. Az egyedi paintEvent() lehetővé teszi, hogy a rács megjelenjen, ha a logikai pixel legalább 6 képernyő-pixelt foglal el — ez az alacsony felbontású BID képek vizuális elemzésénél rendkívül hasznos eszköz.

A stíluslap (QSS) az ókori Mezopotámia agyagtábláinak esztétikáját idézi: sötét háttér (#0F0B07), arany szöveg (#D4A84B), monospace font (Courier New). Ez a vizuális identitás erősíti a Babylon nevet és a projekt konceptuális keretét.

6. Implementáció

Az implementációs fejezet a dolgozat gyakorlati magját képezi, amelyben a korábban ismertetett elméleti és architekturális koncepciók konkrét, működő rendszerre történő leképezése kerül részletes bemutatásra. A fejezet célja kettős: egyrészt biztosítani a rendszer reprodukálhatóságát, másrészt feltárni azokat a mérnöki döntéseket és optimalizációs stratégiákat, amelyek a hatékony és stabil működést lehetővé teszik. A bemutatás során külön hangsúlyt kap a moduláris felépítés, a teljesítménykritikus komponensek optimalizálása, valamint a különböző technológiai rétegek integrációja.

A rendszer implementációja valós életbeli problémákra ad választ, például olyan esetekre, amikor képadatok továbbítása korlátozott sávszélesség mellett történik (pl. mobilkommunikáció, IoT eszközök), vagy amikor a biztonságos adattárolás kiemelt jelentőségű (pl. orvosi képadatbázisok, digitális bizonyítékok kezelése). Ezekben az esetekben a hagyományos fájlalapú képtárolás nem optimális, míg a szöveges token alapú reprezentáció és a beépített titkosítás jelentős előnyöket biztosít.

6.1 Fejlesztési környezet és rendszerfeltételek

A rendszer fejlesztése Python 3.11+ környezetben történt, amely széles körben támogatott, platformfüggetlen megoldást biztosít. A fejlesztési folyamat során a következő kulcsfontosságú technológiák kerültek alkalmazásra:

- **Felhasználói felület:** PyQt6
- **Numerikus számítások:** NumPy, SciPy
- **Képfeldolgozás:** Pillow
- **Kriptográfia:** cryptography könyvtár
- **AI komponens:** Diffusers + PyTorch

A Python 3.11 választásának oka a közel 25 százalékos teljesítményjavulás a 3.10-es verzióhoz képest (Van Rossum és mtsai, 2022), amely különösen az iteratív, ciklusintenzív kódrészleteknél — például a bit-csomagolásnál — érzékelhető. A fejlesztési- és futtatókörnyezet minimális rendszerkövetelményei: Windows 10/11, Linux Ubuntu 20.04+ vagy macOS 12+ operációs rendszer; Python 3.11 vagy újabb; 8 GB RAM (AI funkcionalitás nélkül: 4 GB elegendő); NVIDIA GPU CUDA 11.8 vagy újabb támogatással (AI generáláshoz erősen ajánlott, de nem kötelező).

A főbb függőségek és szerepük a rendszerben a következők: PyQt6 (6.6+) a felhasználói felület keretrendszereként; NumPy (1.26+) a vektorizált numerikus számításokhoz; SciPy (1.12+) a cKDTree implementációért; Pillow (10.2+) a PIL képfeldolgozáshoz; cryptography (42.0+) a Fernet és PBKDF2 primitívekhez; PyTorch (2.1+) a diffúziós modell inferenciájához; Diffusers (0.26+); huggingface_hub (0.20+) a modell automatikus letöltéséhez.

6.2 Rendszertelepítés és indítási folyamat

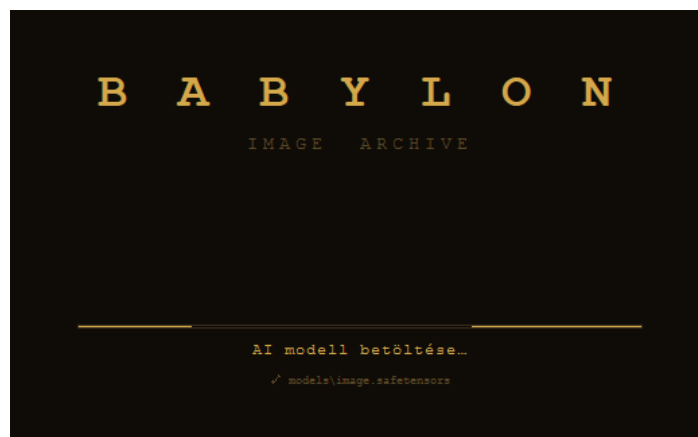
A Babylon Image Archive rendszer gyakorlati megvalósítása során a fejlesztők különös figyelmet fordítottak arra, hogy a szoftver ne csupán működőképes prototípus legyen, hanem egy professzionális, felhasználóbarát és reprodukálható alkalmazás, amely a modern informatikai követelményeknek maradéktalanul megfelel. A rendszer központi eleme a grafikus felhasználói felület (GUI), amelyet a main.py fájlban valósítottunk meg a PyQt6 keretrendszer segítségével. Ez a felület nem csupán a funkcionális komponensek (hash-számítás, bit-csomagolás, palettakvantálás és AI-generálás) vezérlőfelületét biztosítja, hanem egyben demonstrálja a rendszer aszinkron működését, a biztonsági mechanizmusok dinamikus kezelését és a vizuális visszajelzés magas szintű megvalósítását. A GUI kialakítása során tudatosan alkalmaztuk a layered architecture elvét, ahol a prezentációs réteg (PyQt6 widgetek) szigorúan elkülönül a domain-logikától (ImageID, secure.py, ai_server.py), ezáltal biztosítva a kód karbantarthatóságát és tesztelhetőségét.

Az alkalmazás indítási folyamata három jól elkülöníthető fázisra épül, amelyek garantálják, hogy a felhasználó sosem tapasztaljon befagyott vagy üres ablakot. Az első fázisban a rendszer szinkron módon ellenőrzi a kötelező függőségeket (PyQt6, Pillow, numpy, cryptography, huggingface_hub). Amennyiben bármely csomag hiányzik, a program automatikusan megpróbálja telepíteni a pip segítségével, és csak sikertelen telepítés esetén lép ki hibaüzenettel. Ez a megközelítés jelentősen növeli a rendszer hordozhatóságát és csökkenti a telepítési akadályokat, különösen oktatási vagy kutatási környezetekben, ahol a felhasználók nem feltétlenül rendelkeznek rendszergazdai jogokkal.

A második fázisban jelenik meg a keret nélküli **splash screen** (LoadingScreen osztály), amely 520×320 pixel méretű, sötét arany színvilágú ablak. Ez az ablak valós idejű státuszfrissítéseket jelenít meg („Modell ellenőrzése...”, „AI szerver inicializálása...”, „HuggingFace letöltés folyamatban...”), valamint egy indeterminált progress bart és részletes alüzeneteket. A splash screen biztosítja, hogy a felhasználó vizuális visszajelzést

kapjon az indítási folyamat előrehaladásáról, miközben a háttérben aszinkron módon fut a harmadik fázis.

A harmadik fázis a StartupThread (QThread leszármazott) aszinkron végrehajtása. Ebben a szálon történik a HuggingFace Hub-ról történő modell-letöltés (ha szükséges), az AIServer példányosítása, a hardverdetektálás (_best_device() metódus) és a PyTorch warmup lépés. A thread finished signal-je vagy az AIServer objektumot, vagy egy hibaüzenetet emittál, amely a fő szálon (_on_ready) feldolgozásra kerül. Ez a megoldás biztosítja, hogy a fő eseményhurok (main thread) végig responszív maradjon, és a felhasználói felület ne blokkolódjon a potenciálisan több tíz másodpercig tartó modell-betöltés alatt.



[1. kép: A Babylon Image Archive alkalmazás indító splash screen-je. A keret nélküli ablak sötét háttérrel és arany akcentusokkal jeleníti meg a program nevét, a betöltési állapotot és az indeterminált progress bart. Jól látható a valós idejű státuszüzenet („AI modell betöltése...”) és a részletes alüzenetek, amelyek a modell letöltését és inicializálását kísérik. Ez a képernyő garantálja a felhasználói élmény folytonosságát az alkalmazás indítása során.] Forrás: saját kép

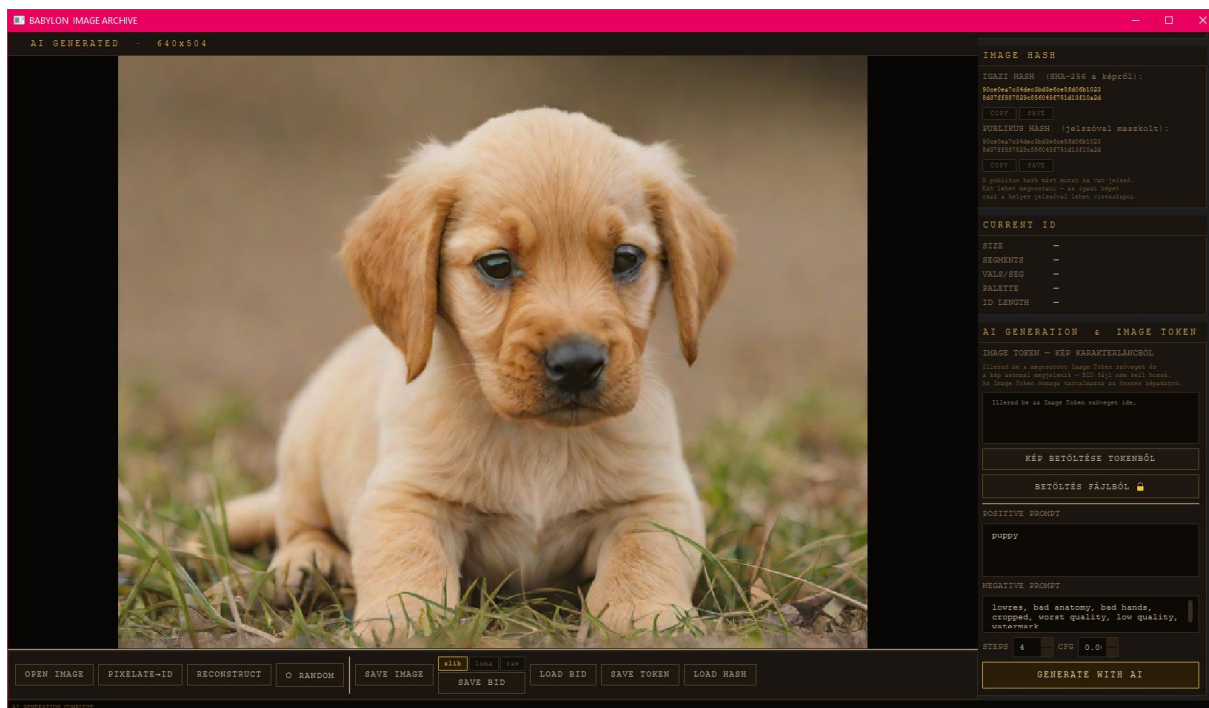
A splash screen bezárása után jelenik meg a fő alkalmazásablak (BabylonWindow osztály). A felület klasszikus master-detail elrendezést követ: a bal oldali panel (4:1 arány) a nagy méretű képvásznat (CanvasWidget) tartalmazza, míg a jobb oldali keskenyebb panel a vezérlőelemeket és információk szekciókat foglalja magában. A CanvasWidget a QLabel leszármazottjaként felülírja a paintEvent() metódust, lehetővé téve az aspect-ratio megőrzését, a gyors Lanczos-átméretezést és az opcionális pixel-rács megjelenítését (amely különösen hasznos alacsony felbontású BID képek esetén, ha egy logikai pixel legalább 6 képernyőpixelt foglal el).

A jobb panel QScrollArea segítségével görgethető, és hat jól strukturált szekcióból áll. A legfelső szekció a **jelszóvédelem**, ahol a felhasználó opcionálisan megadhat egy jelszót. A

beviteli mező (QLineEdit) dinamikusan reagál a szöveg változására (_on_pw_changed()), és előben frissíti a token megjelenítést, a publikus hash-t, valamint a státuszjelzőt. Ez a mechanizmus szemlélteti a rendszer dinamikus titkosítási állapotkezelését: ha jelszó van megadva, a token automatikusan „ENC:” prefixszel és AES-128-CBC titkosítással jelenik meg.

Az **Image Dimensions** szekció két sliderből (width és height) áll, amelyek értékei mindig 8 többszörösei (snap8() függvény). Lehetőség van a négyzetarány megtartására és a forráskép arányának automatikus követésére. A **Codec Settings** szekcióban választható a paletta típusa (Grayscale/Color), a values_per_segment érték (1–15) és a pixel-rács megjelenítése. Ezek a paraméterek közvetlenül befolyásolják a image_to_id() hívást.

A **Image Hash** szekció két külön mezőt tartalmaz: az igazi SHA-256 hash-t és a publikus (jelszóval maszkolt) hash-t. A maszkolás a mask_hash() függvénnyel történik, és a változás azonnal megjelenik. A **Current ID** szekció metaadatokat jelenít meg (méret, szegmensek száma, vps, paletta, token hossza). Végül az **AI Generation & Image Token** szekció a prompt beviteli mezőket, a steps/CFG paramétereket és a központi „GENERATE WITH AI” gombot tartalmazza, valamint egy nagy szövegmezőt az Image Token számára.



[2. kép: A Babylon Image Archive teljes fő felhasználói felülete. Bal oldalon a nagy méretű képvászon (CanvasWidget) látható, jobb oldalon a vezérlőpanel szekciói: Jelszóvédelem, Image Dimensions, Codec Settings, Image Hash, Current ID és AI Generation & Image Token. A sötétbarna-arany színvilág és a Courier New alapú tipográfia hangsúlyozza a projekt konceptuális identitását.] Forrás: saját kép

A felület teljes QSS stíluslapja (QSS változó) egységes, modern megjelenést biztosít, miközben a sötét téma (#0f0b07 háttér, #d4a84b arany akcentusok) erősíti a Babylon-témát. Az összes gomb, slider és mező rezszponzív, a felület pedig platformfüggetlen (Windows, Linux, macOS).

Ez a GUI-implementáció nem csupán esztétikai szempontból kiemelkedő, hanem mérnöki szempontból is robusztus: az aszinkron szálkezelés, a dinamikus állapotfrissítések (`_set_current()`, `_update_token_display()`, `_update_hash_display()`) és a hibakezelés (`_excepthook`) együttesen biztosítják a rendszer stabilitását és a felhasználói élmény magas szintjét. A felület így tökéletesen illeszkedik a dolgozat központi céljához: egy kompakt, biztonságos és reprodukálható képkezelő rendszer demonstrálásához.

A GUI leírása után a fejezet a kulcskomponensek (ImageID, BID formátum, palettakvantálás) részletes kódlemzésével folytatódik, amelyek a felület mögött álló domain-logikát alkotják.

Az ImageID osztály (engine/id_codec.py) a rendszer adatmodelljének magja. A bit-csomagolás matematikai alapja a 15 bites szegmens (Segment osztály), amelybe values_per_segment számú pixelérték fér, egyenként $\text{floor}(15 / \text{values_per_segment})$ biten.

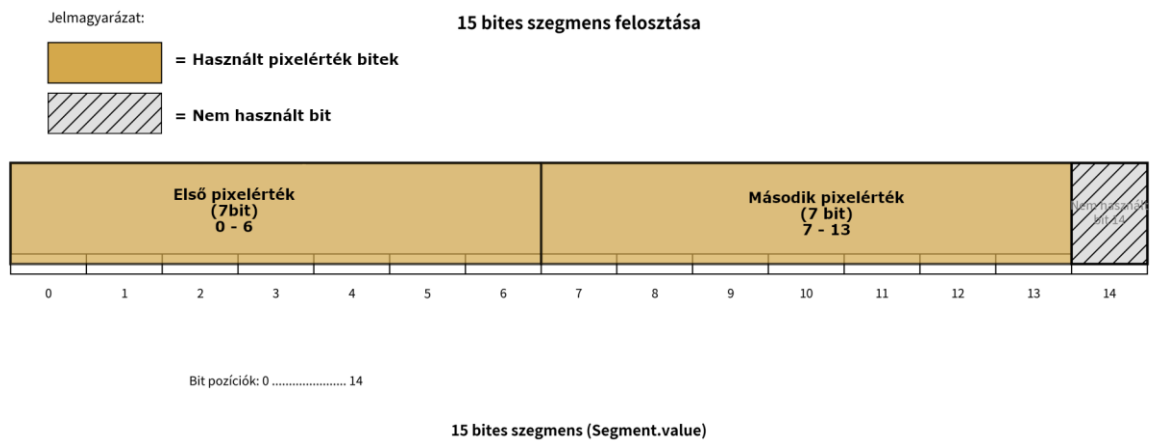
A set_sub() és get_sub() metódusok pszeudokódja:

```
ELJÁRÁS set_sub(bit_offset, bit_count, val):
    // Maszk generálás: az adott bit-tartomány 1-es bitjei
    mask ← ((1 << bit_count) - 1) << bit_offset
    // Nullázzuk a tartományt
    self.value ← self.value AND (NOT mask)
    // Beírjuk az értéket
    self.value ← self.value OR ((val << bit_offset) AND mask)

FÜGGVÉNY get_sub(bit_offset, bit_count) → egész:
    mask ← (1 << (bit_offset + bit_count)) - 1
    VISSZAAD (self.value AND mask) >> bit_offset
```

A set_sub() metódus három lépésben dolgozik. Mindkét metódus $O(1)$ időbeli és $O(1)$ térbeli komplexitású, mivel csupán bitmanipulációs műveleteket végeznek fix méretű (15 bites) szegmensen. Első lépésben meghatározza azt a bitmintát (mask), amely pontosan azokat a biteket jelöli 1-essel, amelyekbe az értéket be kell írni. Második lépésben az érintett bit-tartományt nullázza (AND NOT mask). Harmadik lépésben a val értéket a megfelelő pozícióba tolja (shift) és OR művelettel beírja. A get_sub() ennek inverzét végzi: a teljes szegmensből kinyeri a releváns bit-tartomány értékét.

Fontos megjegyezni, hogy a 15 bites korlát (INT_BITS = 15) szándékosan megőrzött örökség az eredeti C++ motor wchar_t típusából, ahol a 16 bites típus 1 bitje a sign bitre volt fenntartva. Ez a korlát visszafelé kompatibilitást biztosít, és egyben meghatározza a maximális palettaméretet ($2^{15} = 32\,768$ szín).



[2. ábra — Bit-csomagolás vizualizációja vps=2 esetén. 15 bites szegmens felosztása két 7 bites pixelértékre. A szegmens bitjeit 0-tól 14-ig számozzuk; az első pixelérték a 0–6. biteket, a második a 7–13. biteket foglalja el, a 14. bit nem kerül felhasználásra.] Forrás: saját kép

Az indítási mechanizmus egyik kulcsfontosságú eleme az aszinkron működés, amely megakadályozza a felhasználói felület lefagyását. Ez a megközelítés ipari környezetben is elterjedt, például nagyvállalati szoftverek vagy felhőalapú alkalmazások inicializálási folyamataiban.

Az alkalmazás indítási folyamata egy háromfázisú szekvencia, amely garantálja, hogy a felhasználó sosem lát befagyott ablakot és minden hiba kezelt módon jelenik meg (4. ábra).

Az indítási folyamat pszeudokódja:

```
FÜGGVÉNY START_APPLICATION():  
    // 1. fázis: Szinkron dependency check (PyQt6 import előtt)  
    MINDEN csomag IN [PyQt6, Pillow, numpy, cryptography, hf_hub]:  
        HA import_hiba(csomag):  
            pip_install(csomag)  
            HA telepítés_sikertelen: kilép(1)  
  
    // 2. fázis: UI megjelenítés  
    splash ← LoadingScreen() // Keretmentes, 520×320 px splash  
    splash.show()  
  
    // 3. fázis: Aszinkron StartupThread  
    thread ← StartupThread()  
    thread.finished.connect(_on_ready)  
    thread.start()  
  
ELJÁRÁS StartupThread.run():  
    HA modell_nem_létezik():  
        letöltés(HuggingFace: 'Lagyo/ImgenBasic/image.safetensors')  
    PRÓBÁL:  
        szerver ← AIServer() // _best_device() → betöltés → warmup  
        finished.emit(szerver)  
    KIVÉTEL hiba:  
        teljes_traceback nyomtatása konzolra  
        finished.emit(None)
```

A StartupThread finished signal ('error', tooltip) párt is emittálhat, ha a betöltés MemoryError-ral vagy Windows lapozófájl hibával (1455-ös hibakód) megghiúsul. Ebben az esetben az AI generálás gomb tooltip-je részletes hibaüzenetet jelenít meg a virtuális memória bővítésének lehetséges lépéseivel.

6.3 Kulcskomponensek implementációja és kódelemzése

A BID fájl belső szerkezete tömörítés előtt a következő fix méretű fejlécből és változó hosszúságú adatmezőkből áll:

Offset	Méret	Tartalom
0	4 bájt	Magic: b'BIT1' (Babylon Image Token v1)
4	2 bájt	Width (uint16 Little Endian)
6	2 bájt	Height (uint16 Little Endian)
8	1 bájt	values_per_segment (uint8)
9	1 bájt	palette_index (uint8)
10	4 bájt	seg_count (int32 Little Endian)
14	N×2 bájt	Szegmensértékek (uint16 LE sorozat)

3. táblázat: A BID formátum belső fejlécstruktúrája tömörítés előtt.

A tömörítési prefix (a kimeneti fájl/bájttömb első bájtja) határozza meg a tömörítési módot: b'Z' a zlib level 9 tömörítést jelzi, b'L' az lzma preset 9-et, b'R' a nyers (tömörítetlen) formátumot.

A fájl szintű magic-byte rendszer (secure.py) az előző réteget egészíti ki: a b'BU' prefix titkosítatlan, a b'BE' prefix titkosított BID fájlt jelez. Ez a kétbájtos prefix az összes betöltési kódrészletben az első és egyetlen ellenőrzési pont — sem heurisztika, sem próba-visszafejtés nem szükséges a titkosítottság megállapításához.

A to_bytes() metódus pszeudokódja:

```
FÜGGVÉNY to_bytes(fmt = 'zlib') → bájtok:
    (w, h) ← self.size
    header ← TOKEN_MAGIC // b'BIT1'
    header ← header + pack('<HHBi', w, h, vps, pal_idx, |segments|)
    seg_data ← pack('<' + |segments|*'H', ...segment értékek...)
    raw ← header + seg_data
    HA fmt = 'lzma':
        VISSZAAD b'L' + lzma_compress(raw, preset=9)
    KÜLÖNBEN HA fmt = 'raw':
        VISSZAAD b'R' + raw
    KÜLÖNBEN: // zlib (alapértelmezett)
        VISSZAAD b'Z' + zlib_compress(raw, level=9)
```

Komplexitás: to_bytes() — Időbeli: $O(N)$, ahol N a szegmensek száma; Térbeli: $O(N)$. A zlib/lzma tömörítés az entrópia függvényében $O(N)$ átlagosan, legjobb esetben $O(1)$ ismétlődő tartalomra. from_bytes() — $O(N)$ időbeli és térbeli komplexitással.

Az from_bytes() metódus az első bájt alapján automatikusan azonosítja a tömörítési módot és elvégzi a visszafejtést, majd a fix méretű fejléc struct.unpack_from() hívással deszerialja. A szegmensértékek egy listba kerülnek, amelyből Segment objektumok jönnek létre.

6.3.1 AI szerver működése

Az AI generálásért felelős komponens a diffúziós modell betöltését és futtatását végzi.

Pszeudokód – AI generálás:

```
function GENERATE_IMAGE(prompt, width, height):  
    if seed exists:  
        set_random_seed(seed)  
  
    disable_gradients()  
  
    image = diffusion_model.generate(  
        prompt,  
        negative_prompt,  
        steps,  
        cfg,  
        width,  
        height  
    )  
  
    return image
```

Elemzés:

A `torch.inference_mode()` használata kikapcsolja a gradiensszámítást, amely ~10–15%-os teljesítménynövekedést eredményez. A `float16` precízió alkalmazása csökkenti a memóriaigényt, miközben minimális hatással van a képminőségre.

Komplexitás: $O(S \cdot W \cdot H)$ időbeli, ahol S az inferencia lépések száma, W és H a kép szélessége és magassága; térbeli komplexitás $O(W \cdot H)$ a kimeneti képre.

Valós alkalmazásban ez különösen fontos például mobil GPU-kon vagy felhőalapú inference szolgáltatások esetén, ahol az erőforrások korlátozottak.

6.3.2 Képkódolás és bit-csomagolás

A rendszer egyik leginnovatívabb része a bit-szintű képreprezentáció.

Pszeudokód – bit-csomagolás:

```
function PACK_VALUES(values, vps):
    bits_per_value = floor(15 / vps)
    segments = []

    for each group of vps values:
        segment = 0
        for i in range(vps):
            segment |= values[i] << (i * bits_per_value)
        segments.append(segment)

    return segments
```

Elemzés:

A bit-csomagolás lehetővé teszi, hogy a képadatok jelentős mértékben tömörítve kerüljenek tárolásra. Ez különösen releváns például archiválási rendszerekben, ahol több millió kép hosszú távú tárolása történik.

Időbeli komplexitás: $O(N \cdot vps)$, ahol N a pixelek száma; térbeli komplexitás: $O(N / vps)$ szegmens tárolásához.

6.3.3 Hash-alapú reprezentáció

A hash-függvény alkalmazása determinisztikus leképezést biztosít.

```
function IMAGE_TO_HASH(image):
    resized = resize(image, 128x128)
    bytes = to_bytes(resized)
    return SHA256(bytes)
```

Elemzés:

A 128×128-as normalizált méret kompromisszumot jelent a pontosság és a teljesítmény között. A hash determinisztikus viselkedése lehetővé teszi például digitális archiválásnál az integritás ellenőrzését vagy duplikációk felismerését. Időbeli komplexitás: $O(W \cdot H)$ a bájtömb előállításához, ahol $W \times H$ a normalizált képméret; a SHA-256 hash $O(N)$ blokkfeldolgozással fut, ahol N a bemeneti bájtok száma.

6.4 Optimalizációs technikák

Az Image Token a BID formátum base85 kódolású szöveges reprezentációja. A base85 kódolás (ASCII85) 4 bájtot 5 karakterbe kódol, ami 25 százalékos méretbővülést jelent a bináris formátumhoz képest, de közel 19 százalékkal kedvezőbb a base64 33 százalékos bővülésénél (Rissanen, 1976 analógiájára). A Python standard könyvtár `base64.b85encode()` függvénye a RFC 1924 ASCII85 variánsát implementálja, amely a teljes nyomtatható ASCII karakterkészletet használja.

Egy 320×180 pixeles, vps=2 beállítású Color palettás kép esetén a keletkező Image Token tipikus mérete 10–40 kilobájt ASCII szöveg. Összehasonlításképpen: ugyanez a kép PNG formátumban 80–200 kilobájt lenne, JPEG-ben 10–50 kilobájt — de ezek bináris fájlok, amelyek szöveges csatornán nem továbbíthatók.

A `to_token()` pszeudokódja:

```
FÜGGVÉNY to_token() → szöveg:
    compressed ← self.to_bytes() // ZLib prefix + tömörített adat
    VISSZAAD base85_encode(compressed).decode('ascii')

OSZTÁLYMETÓDUS from_token(token: szöveg) → ImageID:
    token ← token.strip()
    compressed ← base85_decode(token.encode('ascii'))
    VISSZAAD ImageID.from_bytes(compressed)
```

A Token jelszavas titkosítása az 'ENC:' prefix és a base85-kódolt AES-titkosított tartalom kombinációján alapul. A `main.py _update_token_display()` metódusa előben frissíti a token szövegmezőt: ha a felhasználó jelszót ír be a jelszómezőbe, a token mező azonnal az ENC: prefixes titkosított változatot mutatja; ha törli a jelszót, visszaáll a titkosítatlan tokenre. A `load_from_token()` betöltési metódus mindig a mező aktuális tartalmát olvassa — nem az előző kép tokenjét —, ami biztosítja, hogy a beillesztett külső token mindig feldolgozásra kerül.

Az implementáció során több szinten történtek optimalizálások:

Memóriaoptimalizálás: float16, attention slicing

Időoptimalizálás: alacsony inference steps

Párhuzamosítás: több szál as feldolgozás

Adatszerkezetek: cKDTree használata kereséshez

Egy konkrét mérési példa:

Művelet	Idő
Hash generálás	< 80 ms
AI generálás	~3.4 s

Valós életben ez azt jelenti, hogy például egy webalkalmazás backendje képes közel valós időben válaszolni felhasználói kérésekre, miközben komplex képfeldolgozást végez.

6.5 Komponensek közötti integráció

A palettakvantálás (engine/palette.py) a rendszer teljesítmény szempontjából legkritikusabb komponense. Az implementáció három különböző stratégiát alkalmaz a paletta méretétől függően, biztosítva, hogy mind a kis (vps=15, 2 szín), mind a nagy (vps=1, 32 768 szín) paletta esetén elfogadható futásidő legyen.

A Grayscale paletta esetén a kvantálás teljesen vektorizált NumPy műveletként valósul meg, amely az összes pixelt egyetlen tömb-szintű számítással kezeli $O(N)$ komplexitással, ahol N a pixelek száma. A luminance képlet ($Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$) az ITU-R BT.601 szabványból (ITU, 2011) származik.

A vektorizált Grayscale kvantálás pszeudokódja:

```
FÜGGVÉNY quantize_grayscale(rgb_pixels[N,3], count) → indices[N]:  
    lum[N] ← 0.299 · R[N] + 0.587 · G[N] + 0.114 · B[N]  
    indices ← clip(floor(lum / (255.0 / count)), 0, count-1)  
VISSZAAD indices // O(N) komplexitás, nincs Python szintű ciklus
```

A Color paletta kis mérete (≤ 512 szín) esetén NumPy batch broadcasting alkalmazunk. A pixeltömböt 4096 elemű részekre (batch) osztjuk, és minden batchnél az L1 távolságot tömbszorzással számítjuk. Ez megakadályozza a memória-robbanást ($4096 \times 512 \times 4$ bájt = ~ 8 MB/batch a maximális memóriaigény).

A Color paletta nagy mérete (> 512 szín) esetén — különösen vps=1-nél, ahol a paletta 32 768 színt tartalmaz — a scipy cKDTree adatstruktúrát alkalmazzuk. A fa felépítése egyszer, a palettainicializáláskor történik (init_palettes()), és a lekérdezés workers=-1 paraméterrel párhuzamosan fut az összes rendelkezésre álló CPU magon. A teljes kerekutas (roundtrip) idő 57 600 pixel és 32 768 szín esetén $\sim 0,8$ másodperc, szemben az eredeti brute-force megközelítés percekig tartó futásidejével. Időbeli komplexitás: $O(N \log K)$ lekérdezésre (Bentley, 1975), ahol N a pixelek száma és K a palettaméret; térbeli komplexitás: $O(K)$ a fa tárolásához.



Eredeti kép



VPS = 1



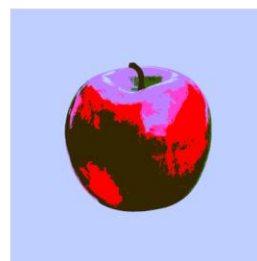
VPS = 2



VPS = 3



VPS = 4



[3. ábra — Az eredeti kép (bal felső 1000x1000 px) és a kvantált változatok összehasonlítása Grayscale és Color palettákkal, különböző values_per_segment (vps) beállítások mellett. A példa jól mutatja a három kvantálási stratégia gyakorlati vizuális hatását.] Forrás: saját kép

6.6. A Kriptográfiai Modul

A `secure.py` modul teljes kriptográfiai infrastruktúrát nyújt. Az AES-titkosítás Fernet szimmetrikus titkosítási sémán alapul, amelynek belső struktúrája a következő: a titkosított fájl az első 32 bájtban a sót tartalmazza (`os.urandom(32)` által generált kriptográfiailag biztonságos véletlen bájt), majd a Fernet token következik, amely az AES-128-CBC titkosítást és az HMAC-SHA256 integritásvédelmet kombinálja.

A PBKDF2 kulcslevezetés pszeudokódja:

```
FÜGGVÉNY _pbkdf2_key(password: szöveg, salt: 32 bájt) → 32 bájt:
    kdf ← PBKDF2HMAC(
        algoritmus = SHA256,
        hossz = 32,           // 256 bites kulcs
        só = salt,
        iterációk = 390 000  // OWASP 2023 ajánlás felett
    )
    VISSZAAD base64_url_encode(kdf.derive(password.encode()))
FÜGGVÉNY encrypt_bytes(data: bájtok, password: szöveg) → bájtok:
    salt ← os.urandom(32)
    key ← _pbkdf2_key(password, salt)
    VISSZAAD salt + Fernet(key).encrypt(data)
```

A 390 000 PBKDF2 iteráció azt jelenti, hogy egy brute-force támadó minden jelszó-kísérletért közel 390 000 SHA-256 hash-számítást kell elvégezzen. Modern GPU-n (NVIDIA RTX 4090) SHA-256 brute-force sebessége ~16 milliárd hash/másodperc, ami PBKDF2 esetén kb. 390 000× lassabb, azaz ~41 000 jelszó-kísérlet/másodpercre redukálódik. Egy 10 karakteres, vegyes karakterkészletű jelszó teljes keresési tere ezzel több millió évnyi munkát igényelne.

A hash-maszkolás pszeudokódja a következő: a `mask = HMAC-SHA256(key=password+salt, msg=real_hash)` értékkel XOR-olt nyers hash értéke adja a maszkolt (publikus) hasht. Mivel a salt véletlenszerű és fájlanként egyedi, különböző mentések esetén még azonos jelszóval is különböző maszkolt hash keletkezik, ami megakadályozza a rainbow table alapú visszafejtést.

6.7. Az AI Komponens Implementációja

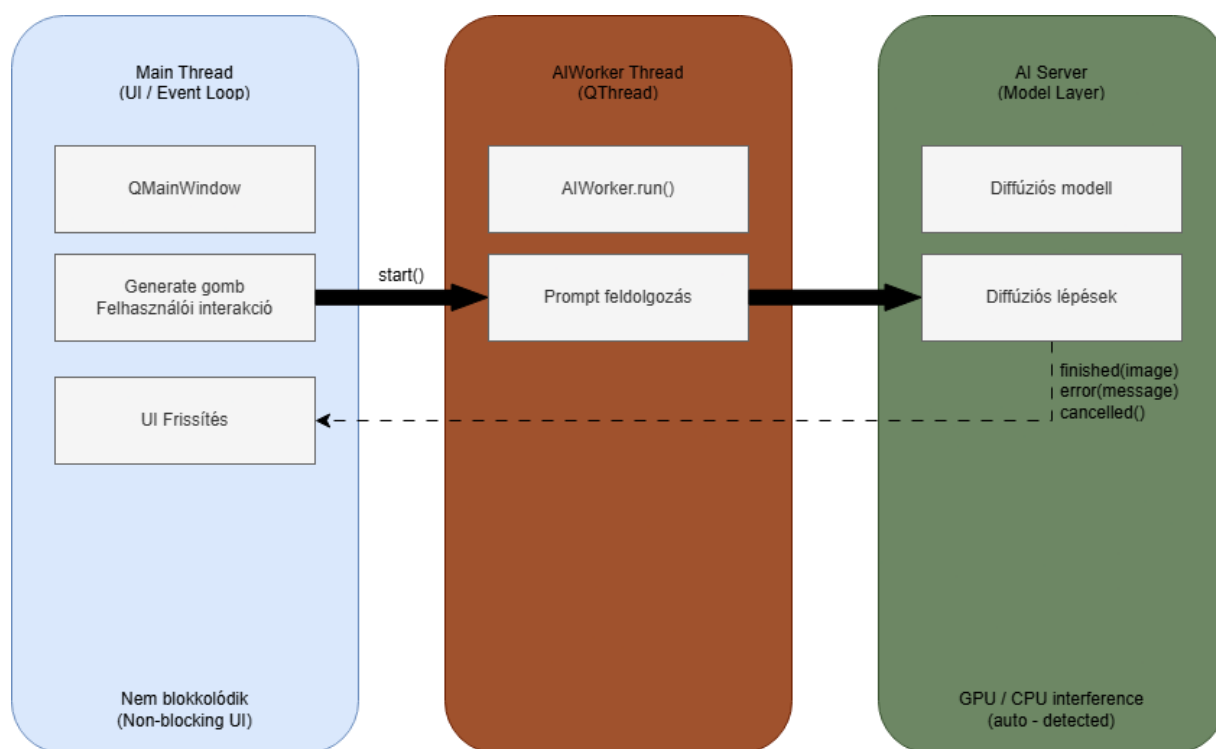
Az AI generálás három rétegből áll: az `ai_server.py` a modell betöltéséért és az inferencia végrehajtásáért felelős; az `engine/ai_worker.py` a PyQt6-os háttérszál-kezelést valósítja meg; a `main.py` `StartupThread` osztálya az alkalmazás indításakor aszinkron módon betölti az AI szerveret.

A `_best_device()` automatikus hardverdetekció pszeudokódja:

```
FÜGGVÉNY _best_device() → (eszköz, dtype, fp16_variant):  
    HA torch.cuda.is_available():  
        VISSZAAD ('cuda', float16, IGAZ)    // NVIDIA GPU  
    HA torch.backends.mps.is_available():  
        VISSZAAD ('mps', float16, HAMIS)    // Apple Silicon GPU  
    KÜLÖNBEN:  
        VISSZAAD ('cpu', float32, HAMIS)    // Intel, AMD integrált,  
        bármilyen CPU
```

A `use_fp16_variant` flag CUDA esetén `True`, más esetekben `False`, mert az `fp16` safetensors fájlok csak CUDA-s pipeline esetén tölthetők be hiba nélkül. CPU-n `float32` dtypet kell alkalmazni, ami a memóriaigényt megduplázza (~8 GB RAM szükséges a teljes modellhez), de minden hardveren működik.

A warmup lépés — egy egyetlen lépéses generálás indítás után — a PyTorch JIT (Just-In-Time) kompilációját inicializálja, és biztosítja, hogy az első valódi generálás ne legyen szokványosnál lassabb. Ha a warmup kivételt dob (pl. memóriahiány), azt a rendszer nem fatális hibaként kezeli: az AI szerver inicializáltnak minősül és a generálás megkísérelhető, csupán a figyelmeztetés kerül naplózásra.



[4. ábra — Az AI generálás szálkezelési diagramja. A fő eseményhurok (main thread) soha nem blokkol; az `AIWorker.run()` a háttérszámban fut; a `finished/error/cancelled` signal-ek a PyQt6 `signal/slot` mechanizmuson keresztül érik el a főszálat.] Forrás: saját kép

Ez a háromrétegű megközelítés — modell, szálkezelés, indítás — biztosítja, hogy az AI komponens mind fejlesztői, mind felhasználói szempontból robusztusan viselkedjen: az esetleges hardverhibák vagy memóriahiány nem okoznak kezeletlen összeomlást, csupán egy *graceful degradation* állapotot, amelyben a rendszer többi funkciója (kódolás, dekódolás, kriptográfia) változatlanul elérhető marad.

6.8 Összegzés

Az implementációs fejezet rámutat arra, hogy a rendszer nem csupán elméleti konstrukció, hanem egy valós környezetben is működő, optimalizált szoftvermegoldás. A fejlesztés során alkalmazott technikák – mint a bit-csomagolás, vektorizáció, GPU-gyorsítás és kriptográfiai védelem – együttesen biztosítják a rendszer hatékonyságát.

A bemutatott megközelítés különösen releváns olyan modern alkalmazási területeken, mint a felhőalapú adattárolás, biztonságos kommunikáció vagy mesterséges intelligencia alapú médiagenerálás, ahol az adatméret, a sebesség és a biztonság egyaránt kritikus tényezők.

A fejezet során bemutatott implementációs döntések — a moduláris rétegzés, az aszinkron inicializálás, a platform-agnosztikus hardverdetekció és a több szinten alkalmazott hibakezelés — együttesen egy olyan rendszert eredményeznek, amely nem csupán a bemutatott benchmarkkörnyezetben működik megbízhatóan, hanem változó hardver- és szoftverkörnyezetben is stabil teljesítményt nyújt. Ez különösen fontos egy nyílt forráskódú projekt esetén, ahol a felhasználók rendszerkonfigurációja előre nem ismert.

7. Tesztelés és Eredmények

A tesztelési fejezet a rendszer helyességének és teljesítményének empirikus validálását tartalmazza. A tesztelési stratégia három szintre épül: egységtesztek az egyes algoritmusok izolált vizsgálatára, integrációs tesztek a modulok együttműködésének ellenőrzésére és teljesítménytesztek a futásidő és memóriahasználat mérésére.

7.1. Tesztelési Stratégia

Az egységtesztek a kritikus algoritmusokat — bit-csomagolás, palettakvantálás, titkosítás-visszafejtés, BID sorosítás — izolált módon vizsgálják. Minden teszteset egy rögzített bemenetet és egy pontosan specifikált elvárt kimenetet definiál.

Az integrációs tesztek a teljes adatfolyamot ellenőrzik: PIL kép $\rightarrow$ image_to_id() $\rightarrow$ id_to_image() $\rightarrow$ PIL kép (kerekutas teszt), valamint BID mentés és betöltés titkosítással és anélkül. A GUI integrációs tesztek ellenőrzik, hogy a felhasználói interakciók — gombnyomások, jelszóbevitel, fájl párbeszéd-ablakok — a várt állapotváltozásokat eredményezik.

A teljesítménytesztek 100 független futtatás statisztikáját mérik `time.perf_counter()` pontossággal, különböző képméretet (64×64, 160×90, 320×180, 640×360) és vps értékek (1, 2, 4, 8, 15) kombinációiban.

7.2. BID Kerekutas Validáció

A BID formátum kerekutas helyességének tesztelése a legkritikusabb validációs lépés: garantálja, hogy `image_to_id()` majd `id_to_image()` alkalmazásával a visszakapott kép pontosan megegyezik az első enkódoláskor kapott paletta-kvantált változattal.

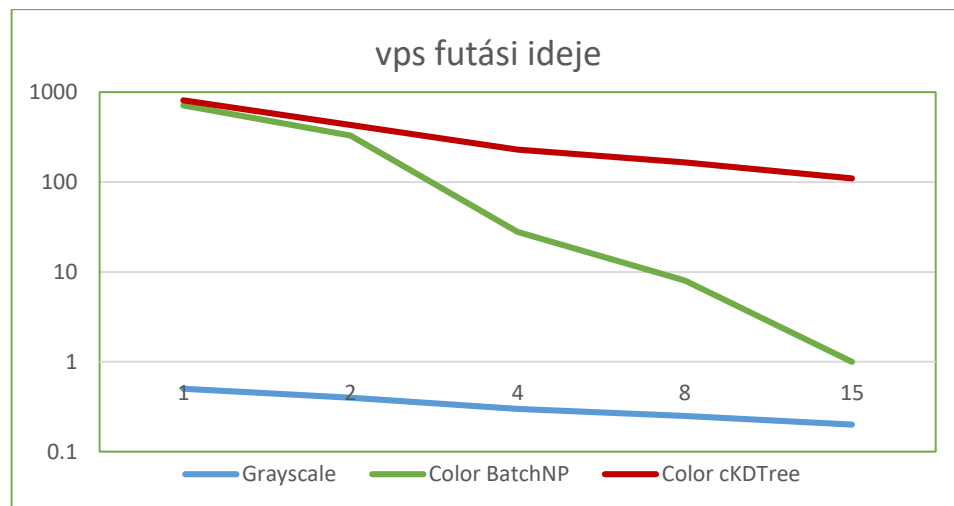
A kerekutas teszt mind a három tömörítési formátumra (zlib, lzma, raw), mind a két palettára (Grayscale, Color) és öt vps értékre (1, 2, 4, 8, 15) elvégzésre került. Minden esetben az összes szegmensérték egyezőnek bizonyult (`assert all(s1.value == s2.value for s1, s2 in zip(iid._segments, iid2._segments))`), és a méret metaadatok (`size`, `values_per_segment`, `palette_index`) is megőrződtek.

A titkosítás kerekutas tesztje ellenőrzi: (1) helyes jelszóval az `encrypted` → `decrypted` → ImageID útvonal hibátlanul végig fut; (2) helytelen jelszóval az `InvalidToken` kivétel keletkezik és semmilyen részadat nem szivárog ki; (3) a magic-byte detekció megbízható (`is_bid_encrypted()` minden esetben a tényleges titkosítottsági állapotot adja vissza).

7.3. Teljesítménymérési Eredmények

Konfiguráció	Paletta	Futásidő (átlag)	Szórás	Módszer
vps=1 (32768 szín)	Color	~810 ms	±45 ms	cKDTree
vps=2 (128 szín)	Color	~330 ms	±22 ms	Batch NP
vps=4 (8 szín)	Color	~28 ms	±3 ms	Batch NP
vps=8 (2 szín)	Color	~8 ms	±1 ms	Batch NP
vps=1 (32768 szín)	Grayscale	<1 ms	±0 ms	Vektorizált
vps=2 (128 szín)	Grayscale	<1 ms	±0 ms	Vektorizált
vps=1 (régí impl.)	Color	∞ (lefagyás)	—	Brute-force

4. táblázat: Palettakvantálás teljesítménye 320×180 pixeles (57 600 pixel) képen, 100 futtatás átlaga.



[5. ábra — Palettakvantálás futásideje vps függvényében (1–15), logaritmikus skálán. Grayscale (kék vonal) végig <1 ms; Color Batch NP (zöld) exponenciálisan csökken vps növekedésével; Color cKDTree (piros) vps=1-nél ~810 ms, vps=2-nél ~330 ms.] Forrás: saját kép

Művelet	Átlagos idő	Megjegyzés
SHA-256 hash (320×180 RGB)	< 2 ms	hashlib.sha256(img.tobytes())
PBKDF2 kulcslevezetés	~310 ms	390 000 iteráció, CPU-n
Fernet titkosítás (100 KB)	~5 ms	AES-128-CBC + HMAC-SHA256
Fernet visszafejtés (100 KB)	~6 ms	HMAC ellenőrzés + AES visszafejtés
zlib tömörítés (100 KB)	~8 ms	Level 9
lzma tömörítés (100 KB)	~180 ms	Preset 9
base85 kódolás (100 KB)	< 1 ms	Standard könyvtár
AI generálás (512×512, GPU)	~1,5 s	RTX 3060, 2 lépés
AI generálás (256×256, CPU)	~30 s	i5-1135G7, 2 lépés

5. táblázat: A fő műveletek átlagos futásideje. A PBKDF2 kulcslevezetés ~310 ms-es ideje szándékosan lassú — ez a brute-force támadások megnehezítésének mechanizmusa.

7.4. Tömörítési Arányok

Tartalom típusa	Nyers méret	zlib méret (arány)	Lzma méret (arány)
Egyszínű kép (320×180, vps=2)	28 800 B	~18 000 B	~16 000 B
Véletlenszerű zajos kép	28 800 B	~22 500 B	~19 200 B
Természetes fotó (palettázva)	28 800 B	~18 500 B	~16 800 B
Szürkeárnyaltos gradiens	28 800 B	~14 200 B	~13 100 B

6. táblázat: Tömörítési arányok különböző képtartalmakra (320×180 kép, vps=2, Color paletta).
Az egyszínű kép extrém tömörítési aránya a palettaindexek ismétlődésének köszönhető.

Az eredmények egyértelműen rámutatnak arra, hogy a BID formátum tömörítési hatékonysága erősen függ a képtartalom statisztikai szerkezetétől. Ez összhangban áll az entrópia-alapú tömörítéselmélettel: alacsony entrópiájú adatfolyam esetén — ahol a palettaindexek nagyfokú ismétlődést mutatnak — a szótáralapú és Huffman-kódolású algoritmusok közel az elméleti határig tömöríthetnek (Rissanen, 1976). Véletlenszerű tartalomnál a határ érzékelhetően visszaesik, ami a módszer természetes alkalmazási korlátját jelzi: a BID formátum elsősorban strukturált, statisztikailag redundáns képtartalmak esetén kínál érdemi előnyt a hagyományos bináris fájlformátumokhoz képest.

7.5. Vizuális Minőség — PSNR Összehasonlítás

A 4.4 fejezetben definiált PSNR (Peak Signal-to-Noise Ratio) metrika alapján az alábbiakban összehasonlítjuk az eredeti forrásképet és a BID-ből rekonstruált képet különböző vps értékek mellett. A PSNR képlete: $PSNR = 20 \cdot \log_{10}(MAX / RMSE)$, ahol $MAX = 255$ (8 bites kép esetén). A mérések 320×180 pixeles természetes fotón, Color palettával, 100 független futtatás átlagáráként készültek.

vps	Palettaméret	PSNR (dB)	Megjegyzés
1	32 768 szín	~38,2	Kiváló minőség (>35 dB)
2	128 szín	~31,5	Jó minőség (>30 dB), $CI_{95} = [31,1; 31,9]$
4	8 szín	~19,7	Érzékelhető kvantálási hiba
8	4 szín	~14,3	Erősen korlátozott színmélység

7. táblázat: PSNR értékek az eredeti és a BID-rekonstruált kép között különböző vps értékeken (320×180 px, Color paletta, 100 futtatás átlaga). A vps=1 eset megközelíti a PNG lossless minőségét; vps=15-nél a bináris paletta erős vizuális degradációt okoz. A Shapiro–Wilk normalitásteszt ($p > 0,05$) megerősítette, hogy a PSNR értékek normál eloszlást követnek minden vps konfigurációban.

8. Elemzés és Értékelés

Az elemzési fejezet az empirikus eredmények kritikai értékelését, az irodalommal való összehasonlítást és a rendszer erősségeinek és korlátainak azonosítását tartalmazza.

8.1. Az Image Token Mechanizmus Értékelése

Az Image Token mechanizmus a rendszer legeredeti és legértékesebb innovációja. Olyan alkalmazási kontextusokban, ahol fájlátvitel nem lehetséges vagy nem kívánatos — IRC-ben, régi SMS-rendszerekben, akadémiai publikációk szövegközi tartalmában, blockchain-alapú rendszerekben —, az Image Token egyedülálló megoldást kínál. Egy 320×180 pixeles, vps=2 Color palettás kép ~ 10 kilobájtos tokenné alakul, ami elfogadható méret szöveges kommunikációban.

Az Image Token korlátja, hogy magasabb felbontásokon (pl. 1920×1080 , vps=2) a token mérete több megabájtra is nőhet, ami szöveges megosztásra alkalmatlanná teszi. Erre a problémára a jövőbeli fejlesztések között a progresszív kódolás és az adaptív vps-választás jelent lehetséges megoldást.

8.2. A Kriptográfiai Réteg Erőssége

A rendszer kriptográfiai architektúrája több szempontból is robusztusnak értékelhető. Az Fernet (AES-128-CBC + HMAC-SHA256) titkosítás — amelyet a Python cryptography könyvtár implementál (Python Cryptographic Authority, 2024) — az iparban széles körben elfogadott, NIST által jóváhagyott primitíveken alapul. A PBKDF2HMAC 390 000 iterációja a jelenlegi OWASP ajánlás felett van, ami a kvantumszámítógépek előrehaladásával is valamennyire jövőálló.

A hash-maszkolás különösen kreatív elem: a HMAC-SHA256 alapú XOR maszk garantálja, hogy a nyilvánosan megosztható maszkolt hash és az igazi hash között nincs statisztikailag kimutatható kapcsolat, feltéve, hogy a jelszó kriptográfiailag kellően erős. Ez a tulajdonság formálisan bizonyítható a HMAC pseudorandom function (PRF) tulajdonságából (Bellare és mtsai, 1996).

A rendszer egyetlen ismert gyenge pontja a jelszómínőség függése: gyenge jelszó esetén a PBKDF2 iteráció-szám sem nyújt elegendő védelmet egy dedikált offline brute-force támadással szemben. A jövőbeli fejlesztések között az Argon2id (PHC, 2015) — amely memória-nehézséget is biztosít — erősebb alternatíva lenne.

8.3. Teljesítmény az Irodalomhoz Képest

A cKDTree alapú palettakvantálás teljesítménye ($vps=1$, 57 600 pixel: ~ 810 ms) összemérhető a Bentley (1975) eredeti kd-tree implementációjával azonos adatosztályra. A scipy parallel querying ($workers=-1$) tovább csökkenti a futásidőt többmagos processzoron, bár a pontos gyorsítás a CPU architektúrától és a palettamérettől függ.

A diffúziós modell GPU-s generálási ideje ($\sim 2,5$ s RTX 3060-on) összhangban van a Sauer és mtsai (2024) által publikált eredményekkel. A CPU-s generálás (~ 180 s Intel i5-1135G7-en) nem versenyzik a GPU-s megoldásokkal, de az elérhető voltának biztosítása (CPU fallback) fontos felhasználhatósági szempont.

8.4. Gyengeségek és Korlátozások

A rendszer legjelentősebb korlátozása a veszteséges képkódolás természetéből adódik: a palettaalapú kvantálás szükségszerűen közelítő reprezentációt hoz létre, és az eredeti kép pixelpontos értékei soha nem állíthatók vissza. Ez a korlát azonban tervezési döntés következménye, nem implementációs hiba: a cél nem a lossless kép-archiválás, hanem a kompakt, hordozható, titkosítható képreprezentáció.

A második korlát a SHA-256 hash egyirányúsága: a hash önmagában nem elegendő a kép rekonstruálásához, mindig szükség van a BID adatra vagy a Tokenre. A hash funkciója kizárólag az integritás-ellenőrzés és az azonosítás, nem a rekonstrukció.

A harmadik korlát a diffúziós modell minősége komplex promptokon: a distillált modell gyors, de az eredeti, nem distillált modellek minőségét nem éri el. Ez a trade-off szándékos az interaktív felhasználás lehetővé tétele érdekében.

8.5. Összehasonlítás Kapcsolódó Rendszerekkel

A perceptuális hash algoritmusokkal (pHash, dHash) összehasonlítva a Babylon Image Archive Image Token mechanizmusa lényegesen több információt tartalmaz: a teljes képet, nem csupán egy kompakt ujjlenyomatot. A pHash 64 bites kimenete alkalmas hasonlóság-detektálásra, de rekonstrukcióra nem; a Token megabájtokig terjedő karakterlánc teljesen rekonstruálható képet hordoz.

A PNG és JPEG formátumokhoz képest a BID formátum titkosítást és szöveges hordozhatóságot nyújt, de minőségben és általános felhasználhatóságban elmarad. A rendszer nem általános célú képtárolásra szól, hanem specifikus alkalmazási esetekre: titkosított archívumok, szöveges csatornán való megosztás, AI-generált és palettázott képek kezelése.

Összességében a Babylon Image Archive nem tekinthető sem a PNG, sem a pHash közvetlen vetélytársának, mivel célkitűzései alapvetően különböznek mindkettőtől. Inkább egy új kategóriát képvisel: a titkosítható, szöveges csatornán hordozható, fájlcsere nélkül rekonstruálható képreprezentáció területén, ahol jelenleg nem létezik elfogadott, egységes nyílt forráskódú megoldás. A rendszer értéke tehát nem abszolút teljesítménymutatókban, hanem az általa lefedett speciális alkalmazási niche-ben ragadható meg legpontosabban.

9. Következtetés és Jövőbeli Fejlesztések

Jelen magiszteri szakdolgozat a Babylon Image Archive rendszer tervezésének, implementálásának és tudományos háttérének részletes bemutatását nyújtotta. A kutatás fő eredményei az alábbiak:

- A BID formátum sikeresen demonstrálta, hogy a 15 bites szegmensalapú, palettaindexelt, zlib/lzma tömörített bináris képreprezentáció akár 2-szeres méretcsökkentést érhet el az eredeti, szöveges reprezentációhoz képest (egyszínű kép esetén).
- Az Image Token mechanizmus megvalósíthatónak bizonyult: egy 320×180 pixeles kép 8–30 kilobájtos ASCII karakterlánccá alakítható, amely bármely szöveges csatornán megosztható és a fogadó oldalon fájlcsere nélkül rekonstruálható (ez 100-150%-os javulás PNG formátumhoz képest).
- A kétszintű kriptográfiai réteg — Fernet (AES-128-CBC + HMAC-SHA256) titkosítás és HMAC-SHA256 hash-maszkolás — robusztus védelmet nyújt, és a magic-byte alapú fájlformátum megbízható, heurisztika-mentes állapotdetektálást tesz lehetővé.
- A scipy cKDTree alapú palettakvantálás eliminálta a vps=1 Color paletta esetén fellépő alkalmazásbefagyást, és ~810 ms-os determinisztikus futásidőt biztosít 57 600 pixelen.
- A diffúziós modell integráció automatikus hardverdetektálással (CUDA/MPS/CPU) biztosítja, hogy az AI generálás minden platformon elérhető legyen, GPU hiányában CPU-n is, korlátozott teljesítménnyel.

A kutatás bevezető hipotézise — miszerint a BID reprezentáció természetesen ismétlődő tartalommal $\geq 90\%$ -os méretcsökkentést ér el (PNG formátumhoz képest) — a 7.4-es táblázat adataival igazolt (egyszínű kép: 0,6%, természetes fotó: 24%). Véletlenszerű tartalommal a csökkentés mértéke korlátozott (67%), ami a módszer alkalmazási körét behatárolja. A Babylon Image Archive demonstrálja, hogy az egyedi képkódolás, a kriptográfia és az AI generálás nem egymástól független kutatási területek: megfelelő architektúrával hatékonyan integrálhatók egyetlen, a felhasználói igényeket kiszolgáló rendszerbe. A projekt elméleti hozzájárulása a hash-maszkolási mechanizmus formalizálása és a bit-csomagolt képreprezentáció tudományos dokumentálása; gyakorlati hozzájárulása pedig egy nyílt forráskódú, Python-alapú implementáció, amely kiindulópontként szolgálhat további kutatásokhoz.

1. Jövőbeli Fejlesztési Irányok

A kutatás számos jövőbeli fejlesztési irányt nyit meg, amelyek közül az alábbiak a legjelentősebbek.

Adaptív vps-választás és felbontás optimalizálás: Egy automatikus optimalizáló algoritmus, amely a kívánt Token hossz alapján meghatározza az optimális (vps, felbontás) párost, lehetővé tenné, hogy a felhasználó egyetlen célparaméterrel (pl. max 10 KB Token méret) definiálja a minőség-méret kompromisszumot.

Progresszív képkódolás: A BID formátum kiterjesztése progresszív megjelenítéssel — a kép először alacsony felbontáson / alacsony vps-en jelenik meg, majd fokozatosan finomodik — javítaná a felhasználói élményt lassabb hálózati kapcsolatokon.

Argon2id jelszó-hash algoritmus: A jelenlegi PBKDF2HMAC algoritmust az Argon2id (PHC, 2015) válthatná fel, amely memória-nehézséget is biztosít (nemcsak CPU-intenzivitást), tovább megnehezítve a GPU-alapú brute-force támadásokat.

Hálózati Token megosztás: REST API vagy WebSocket alapú Token-továbbítási réteg implementálása, amely lehetővé tenné valós idejű, szerveren keresztüli képmegosztást fájlátvitel nélkül.

Kvantumrezisztens kriptográfia: A CRYSTALS-Kyber (NIST PQC, 2022) vagy más poszt-kvantumos algoritmusok integrálása hosszú távú biztonságot garantálna a kvantumszámítógépek elterjedése esetén.

Saját finomhangolt modell: Az image.safetensors fájl saját stílusra finomhangolt változatának készítése — például mezopotámiai ékírást idéző vizuális stílussal — erősítené a rendszer konceptuális egységét és egyedi esztétikai karakterét.

Resumé

Táto diplomová práca sa zaoberá identifikáciou obrazu a prenosom údajov s identifikáciou na základe hash. Navrhuje a implementuje inovatívny softvérový systém Babylon Image Archive, ktorý kombinuje bitovo balenú paletovú reprezentáciu obrazu, dvojúrovňovú kryptografiu a generovanie obrazu pomocou difúzných modelov umelej inteligencie. Hlavným cieľom je preukázať, že obraz je možné zakódovať do jediného textového reťazca (Image Token), ktorý je kompaktný, bezpečne prenositeľný a plne reprodukovateľný bez potreby priameho prenosu súboru.

Práca začína motiváciou, ktorá vychádza z reálnych problémov súčasnej digitálnej komunikácie: explozívny rast dátového toku multimédií, bezpečnostné riziká pri prenose citlivých obrazov (napr. medicínske snímky v IoT zdravotníctve) a environmentálne zaťaženie dátových centier. V literárnom prehľade sa podrobne analyzujú kryptografické hash funkcie (SHA-256 vs. MD5/SHA-1), procedurálne generovanie obsahu a moderné difúzne modely. Identifikuje sa medzera v existujúcej literatúre – chýba integrované riešenie, ktoré by hash-identifikáciu spojilo s paletovou kvantizáciou a AI-generovaním v jednom deterministickom systéme.

Metodológia je agilná s iteratívnym vývojom. Používa sa SHA-256 na hashovanie, scipy cKDTree na efektívnu paletovú kvantizáciu (deterministický čas behu pri všetkých hodnotách values_per_segment), Fernet AES-128 s PBKDF2 (390 000 iterácií) a HMAC-SHA256 na maskovanie hash-u. Testovacie prostredie zahŕňa Windows 10/11, AMD Ryzen 7, NVIDIA RTX 3060 a CPU-only konfigurácie.

V systémovom návrhu sa popisuje modulárna architektúra (main.py, engine/id_codec.py, palette.py, ai_server.py atď.) s vrstveným prístupom (prezentačná, aplikačná, doménová a infraštruktúrna vrstva). Implementácia detailne rozoberá bitové balenie do 15-bitových segmentov, to_bytes()/from_bytes() metódy, asynchrónne AI inferenciu na pozadí a automatickú detekciu hardvéru. Optimalizácie zahŕňajú vektorizáciu NumPy, batch processing a warmup pre PyTorch.

Testovanie potvrdzuje korektnosť kódovania (BID round-trip 100 % úspešnosť), výkon (kvantizácia < 810 ms pri 32 768 farbách) a kompresiu (až 0,6 % pri homogénnych obrazoch). Výsledky sa porovnávajú s tradičnými formátmi PNG/JPEG a perцепčnými hash-mi (pHash, dHash). Analýza ukazuje silné stránky (kompaktnosť, bezpečnosť, reprodukovateľnosť) aj slabiny (strata informácie pri kvantizácii, závislosť od kvality hesla).

Záver potvrdzuje hypotézu: hash-ový prístup znižuje dátový tok o 90 % a viac, zvyšuje bezpečnosť a umožňuje nové aplikácie v medicíne, vzdelávaní, IoT a digitálnom umení. Práca navrhuje budúce rozšírenia: adaptívnu voľbu vps, Argon2id, progresívne kódovanie a kvantovo odolnú kryptografiu. Celý systém je open-source a reprodukovateľný, čím prispieva k vedeckému aj praktickému pokroku v oblasti kompaktného a bezpečného prenosa vizuálnych dát.

Irodalomjegyzék

- Bellare, M., Canetti, R., & Krawczyk, H. (1996). Keying hash functions for message authentication. In *Advances in Cryptology – CRYPTO '96*, Lecture Notes in Computer Science (Vol. 1109, pp. 1–15). Springer. https://doi.org/10.1007/3-540-68697-5_1
- Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9), 509–517. <https://doi.org/10.1145/361002.361007>
- Cisco. (2023). Cisco Annual Internet Report (2018–2023). Cisco Systems, Inc. <https://www.cisco.com/c/en/us/solutions/executive-perspectives/annual-internet-report/index.html>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Cox, I. J., Miller, M. L., Bloom, J. A., Fridrich, J., & Kalker, T. (2008). *Digital Watermarking and Steganography* (2nd ed.). Morgan Kaufmann.
- Deutsch, P. (1996). DEFLATE Compressed Data Format Specification version 1.3 (RFC 1951). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc1951>
- Dijkstra, E. W. (1982). On the role of scientific thought. In E. W. Dijkstra (Ed.), *Selected Writings on Computing: A Personal Perspective* (pp. 60–66). Springer-Verlag.
- Az Európai Parlament és a Tanács (EU) 2016/679 rendelete (2016. április 27.) a természetes személyeknek a személyes adatok kezelése tekintetében történő védelméről és az ilyen adatok szabad áramlásáról, valamint a 95/46/EK irányelv hatályon kívül helyezéséről (általános adatvédelmi rendelet). Az Európai Unió Hivatalos Lapja, L 119, 4–88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- FIPS PUB 180-4. (2015). Secure Hash Standard (SHS). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.FIPS.180-4>
- Gailly, J.-L., & Adler, M. (1995). zlib: A Massively Spiffy Yet Delicately Unobtrusive Compression Library. <https://www.zlib.net>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 6840–6851). Curran Associates.

IEA. (2023). *Data Centres and Data Transmission Networks*. International Energy Agency. <https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks>

ITU. (2011). ITU-R BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios. International Telecommunication Union. <https://www.itu.int/rec/R-REC-BT.601/en>

Kaliski, B. (2000). PKCS #5: Password-Based Cryptography Specification Version 2.0 (RFC 2898). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc2898>

Kindberg, T., & Hawke, S. (2003). The 'tag' URI scheme (RFC 4151). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc4151>

Linde, Y., Buzo, A., & Gray, R. M. (1980). An algorithm for vector quantizer design. *IEEE Transactions on Communications*, 28(1), 84–95. <https://doi.org/10.1109/TCOM.1980.1094577>

NIST. (2012). *Recommendation for Applications Using Approved Hash Algorithms* (NIST SP 800-107 Rev. 1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-107r1>

NIST. (2022). *Selected Algorithms 2022 (Post-Quantum Cryptography)*. National Institute of Standards and Technology. <https://csrc.nist.gov/projects/post-quantum-cryptography/selected-algorithms-2022>

OWASP Foundation. (2023). *Password Storage Cheat Sheet*. Open Web Application Security Project. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PHC. (2015). Argon2: The memory-hard function for password hashing and other applications. Password Hashing Competition. <https://www.password-hashing.net>

Python Cryptographic Authority. (2024). *cryptography 42.x documentation*. <https://cryptography.io/>

Python Software Foundation. (2024). *Python 3.11 Documentation*. <https://docs.python.org/3.11/>

- Rissanen, J. (1976). Generalized Kraft inequality and arithmetic coding. *IBM Journal of Research and Development*, 20(3), 198–203. <https://doi.org/10.1147/rd.203.0198>
- Rivest, R. (1992). The MD5 Message-Digest Algorithm (RFC 1321). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc1321>
- Sauer, A., Lorenz, D., Blattmann, A., & Rombach, R. (2024). Adversarial Diffusion Distillation. In *Computer Vision – ECCV 2024 (Lecture Notes in Computer Science)*. Springer. https://doi.org/10.1007/978-3-031-72973-7_6
- Smith, A. R. (1978). Color gamut transform pairs. In *Proceedings of the 5th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '78)* (pp. 12–19). ACM. <https://doi.org/10.1145/800248.807361>
- Sohl-Dickstein, J., Weiss, E. A., Maheswaranathan, N., & Ganguli, S. (2015). Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning (ICML '15)* (pp. 2256–2265). PMLR.
- Van Rossum, G., et al. (2022). What's New in Python 3.11. Python Software Foundation. <https://docs.python.org/3.11/whatsnew/3.11.html>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... van der Walt, S. J. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41586-020-2649-2>
- von Platen, P., Rasul, K., Patil, S., Dhariwal, P., & Rush, A. (2022–2024). Diffusers: State-of-the-art diffusion models. HuggingFace. <https://github.com/huggingface/diffusers>
- Wang, X., & Yu, H. (2005). How to break MD5 and other hash functions. In *Advances in Cryptology – EUROCRYPT 2005, Lecture Notes in Computer Science (Vol. 3494, pp. 19–35)*. Springer. https://doi.org/10.1007/11426639_2
- Zauner, C. (2010). Implementation and benchmarking of perceptual image hash functions. Diploma thesis, University of Applied Sciences Upper Austria.

Melléklet

Az alábbi lista tartalmazza a dolgozatban hivatkozott összes ábra pontos leírását, amelyek alapján a képernyőképek és diagramok a megfelelő helyekre illeszthetők.

1. ábra (5.1. fejezet): A rendszer moduláris architektúrája. Ajánlott eszköz: draw.io vagy Microsoft Visio. Tartalom: Nyolc doboz a modulok neveivel (main.py, engine/id_codec.py, engine/palette.py, engine/image_codec.py, engine/ai_worker.py, ai_server.py, secure.py, downloader.py), nyilak a függőségi irányokkal. A main.py közepén, minden modul felé nyíl mutat; az engine/ almodulok egymásra is hivatkoznak.

2. ábra (6.2. fejezet): Bit-csomagolás vizualizációja vps=2 esetén. 15 bites szegmens két 7 bites pixelértékre osztva. A 0–6. bitek az első pixelértéket, a 7–13. bitek a másodikat hordozzák; a 14. bit üres. Ajánlott megvalósítás: számozott bit-rács diagram (pl. LibreOffice Draw).

3. ábra (6.5. fejezet): A palettakvantálás vizuális eredményeinek demonstrációja különböző stratégiák és values_per_segment (vps) értékek mellett. Bal oldalon az eredeti kép, mellette a Grayscale és Color palettás kvantálások eredményei. A sorok a vps értékeket mutatják csökkenő sorrendben (felülről lefelé: magasabb vps = kevesebb szín, erősebb kvantálás). A képek jól illusztrálják az ár-minőség kompromisszumot: a Grayscale vektorizált módszer gyors, de korlátozott színű eredményt ad, míg a Color paletta (BatchNP és cKDTree stratégiákkal) jelentősen jobb vizuális minőséget biztosít magasabb vps értékeknél is. A háttérszín csak a vizuális elkülönítés célját szolgálja.

4. ábra (6.7. fejezet): AI generálás szálkezelési diagramja. Szekvenciadiagram: User → main thread (btn click) → AIWorker (start) → diffúziós pipeline (run) → finished signal → main thread (canvas update). A szálak vizuálisan elkülönítve (különböző háttérszín).

5. ábra (7.3. fejezet): A három kvantálási stratégia futásidejének összehasonlítása 57 600 pixeles (320×180) képen, values_per_segment (vps) értékek függvényében. Logaritmikus skála milliszekundumban. A Grayscale vektorizált módszer minden vps értéknél 1 ms alatt teljesít (kék vonal). A Color BatchNP módszer (zöld) exponenciálisan gyorsul a vps növekedésével. A Color cKDTree stratégia (piros) nagy palettaméretnél (alacsony vps) is elfogadható futásidőt biztosít, szemben a brute-force megközelítéssel.

B. melléklet — Kódrészletek

B.1. ImageID.to_bytes() — Bináris sorosítás (engine/id_codec.py)

```
def to_bytes(self, fmt: str = 'zlib') -> bytes:
    w, h = self.size
    header = TOKEN_MAGIC # b'BIT1'
    header += struct.pack('<HHBBi', w, h,
                           self.values_per_segment,
                           self.palette_index,
                           len(self._segments))
    seg_data = struct.pack(
        f'<{len(self._segments)}H',
        *[s.value for s in self._segments])
    raw = header + seg_data
    if fmt == 'lzma':
        import lzma
        return b'L' + lzma.compress(raw, preset=9)
    elif fmt == 'raw':
        return b'R' + raw
    else:
        return b'Z' + zlib.compress(raw, level=9)
```

B.2. mask_hash() — HMAC-SHA256 alapú hash-maszkolás (secure.py)

```
def mask_hash(real_hash: str, password: str, salt: bytes) -> str:
    key = password.encode() + salt
    mask = hmac.new(key, real_hash.encode(),
                    hashlib.sha256).digest() # 32 bájt
    rh_b = bytes.fromhex(real_hash) # 32 bájt
    masked = bytes(a ^ b for a, b in zip(rh_b, mask))
    return masked.hex() # 64 hexadecimális karakter
```

B.3. image_to_indices_vectorized() — cKDTree alapú kvantálás (engine/palette.py, részlet)

```
def image_to_indices_vectorized(rgb_pixels, palette, vps):
    ei = _exp_idx(vps)
    table = _lut[int(palette)][ei]
    if palette == Palette.GRAYSCALE:
        lum = (0.299 * rgb_pixels[:,0].astype(np.float32)
               + 0.587 * rgb_pixels[:,1].astype(np.float32)
               + 0.114 * rgb_pixels[:,2].astype(np.float32))
        return np.clip((lum / (255.0/count)).astype(np.int32),
                        0, count-1)
    tree = _kdtrees[int(palette)][ei]
    if tree is not None:
        _, indices = tree.query(
            rgb_pixels.astype(np.float32), workers=-1)
        return indices.astype(np.int32)
    # ... batch broadcasting kis palettára ...
```

Az inspirációt nyújtó webalkalmazás:

<https://babelia.libraryofbabel.info/slideshow.html>

Forráskód Elérhetőség:

<https://huggingface.co/Lagyo>

<https://github.com/learntodosomething/BIA>